

Lecture 6: Microbiome Analysis with R

高春辉

2026-03-20

Table of contents

前言	5
微生物组研究概述	5
分析流程概览	5
本讲内容组织	5
示例数据说明	6
软件环境	6
核心概念速览	6
ASV 与 OTU	6
microtable 对象	6
组成性效应 (Compositional Effect)	6
使用说明	6
学习建议	7
第 1 章 测序数据预处理	9
1.1 数据准备	9
1.2 分析概述	9
1.3 加载需要的包	9
1.4 检查实验数据	9
1.4.1 生成 metadata	10
1.4.2 查看文件的测序质量	10
1.5 过滤和修剪	11
1.5.1 设置过滤文件的产生路径和名称	11
1.5.2 对原始数据进行过滤	11
1.5.3 估计测序错误率	12
1.5.4 序列去重	14
1.5.5 根据模型进行匹配	14
1.6 ASV 表	14
1.6.1 序列拼接	14
1.6.2 生成 ASV 表	14
1.6.3 选择目标长度的序列	15
1.6.4 去嵌合体	15
1.6.5 测序数据处理情况追踪	16
1.7 分配分类	16
1.7.1 分类注释	16
1.8 保存结果	16
1.8.1 保存 ASV 序列	16
1.8.2 保存 ASV 表和分类表	17
第 2 章 群落结构分析	19
2.1 概述	19
2.1.1 什么是群落结构分析?	19
2.1.2 为何群落结构分析是第一步?	19
2.1.3 microtable 对象: 分析的基础	19
2.1.4 分析内容	20
2.2 流程	20
2.3 实现	20

2.4	构建 microtable 对象	20
2.4.1	载入包	21
2.4.2	OTU 表	21
2.4.3	Sample 表	21
2.4.4	Taxonomy 表	21
2.4.5	构建对象	21
2.4.6	门水平的物种组成	22
2.4.7	纲水平的物种组成	22
2.4.8	属水平的热图	23
2.4.9	箱线图比较组间差异	24
2.5	结果	25
2.5.1	群落组成特征	25
2.5.2	处理组间差异的初步观察	25
2.5.3	热图揭示的精细模式	25
2.5.4	分析与讨论	25
2.6	小提示	26
第 3 章	α 多样性	27
3.1	概述	27
3.1.1	常用 Alpha 多样性指标	27
3.1.2	分析目标	27
3.2	流程	27
3.3	构建 microtable 对象	27
3.3.1	载入包	28
3.3.2	OTU 表	28
3.3.3	Sample 表	28
3.3.4	Taxonomy 表	28
3.3.5	构建对象	28
3.4	实现	29
3.4.1	抽平 (Rarefaction)	29
3.4.2	计算 Alpha 多样性指标	30
3.4.3	创建分析对象并进行差异检验	30
3.4.4	可视化	31
3.5	结果	33
3.5.1	Alpha 多样性总体特征	33
3.5.2	组间差异分析	33
3.5.3	与环境因子的关系 (可选拓展)	34
3.5.4	结论与建议	34
3.6	小提示	35
第 4 章	β 多样性	37
4.1	概述	37
4.1.1	Beta 多样性的生态学意义	37
4.1.2	常用距离/相异度指标	37
4.1.3	分析内容	38
4.2	流程	38
4.3	构建 microtable 对象	38
4.3.1	载入包	39
4.3.2	OTU 表	39
4.3.3	Sample 表	39
4.3.4	Taxonomy 表	39

4.3.5	构建对象	39
4.4	实现	40
4.4.1	计算 Beta 多样性距离矩阵	40
4.4.2	PCoA 分析	40
4.4.3	NMDS 分析	41
4.4.4	组间距离分析	43
4.4.5	PERMANOVA 统计检验	44
4.4.6	ANOSIM 分析	45
4.5	结果	45
4.5.1	排序图揭示的群落结构	45
4.5.2	组间差异的统计分析	45
4.5.3	综合讨论	45
4.6	小提示	46
第 5 章	菌群差异分析	47
5.1	概述	47
5.1.1	差异分析的科学目标	47
5.1.2	常用差异分析方法	47
5.1.3	方法选择建议	47
5.2	流程	47
5.3	构建 microtable 对象	47
5.3.1	载入包	49
5.3.2	OTU 表	49
5.3.3	Sample 表	49
5.3.4	Taxonomy 表	49
5.3.5	构建对象	49
5.4	实现	50
5.4.1	LEfSe 方法	50
5.4.2	随机森林方法	54
5.4.3	ANCOM-BC2 方法	54
5.4.4	其他方法示例	56
5.5	结果解读	57
5.5.1	差异生物标志物的识别	57
5.5.2	可能的生物学发现	57
5.5.3	结果总结表格	57
5.5.4	研究局限性与建议	58
5.6	小提示	58
第 6 章	网络分析	59
6.1	概述	59
6.1.1	网络分析的生态学意义	59
6.1.2	网络构建方法	59
6.1.3	网络拓扑指标	59
6.1.4	节点分类 (基于 Z_i 和 P_i)	59
6.2	流程	59
6.3	构建 microtable 对象	60
6.3.1	载入包	60
6.3.2	OTU 表	60
6.3.3	Sample 表	61
6.3.4	Taxonomy 表	61
6.3.5	构建对象	61

6.4	实现	61
6.4.1	相关性网络构建	61
6.4.2	计算网络	62
6.4.3	模块划分	62
6.4.4	网络可视化	62
6.4.5	网络拓扑分析	63
6.4.6	边和邻接矩阵	66
6.4.7	子网络提取	66
6.4.8	分类群间连接分析	67
6.4.9	其他网络构建方法	67
6.5	结果	68
6.5.1	网络整体特征	68
6.5.2	关键节点识别	68
6.5.3	模块的功能推断	68
6.5.4	网络与处理效应的关系	68
6.5.5	与其他分析的关联	69
6.5.6	研究局限性与注意事项	69
6.6	小提示	69
第 7 章	总结	71
7.1	核心内容回顾	71
7.1.1	测序数据预处理 (DADA2)	71
7.1.2	构建 microtable 对象	71
7.1.3	群落结构分析	71
7.1.4	Alpha 多样性分析	71
7.1.5	Beta 多样性分析	71
7.1.6	差异分析	72
7.1.7	网络分析	72
7.2	分析流程总结图	72
7.3	关键注意事项	72
7.3.1	数据质量	72
7.3.2	统计分析	72
7.3.3	结果解释	73
7.4	拓展学习方向	73
7.5	推荐资源	73
7.5.1	文档与教程	73
7.5.2	重要文献	73
7.5.3	R 包资源	73
7.6	结语	73
References		75

前言

欢迎来到《数据驱动的可重复性研究》课程第六讲：使用 R 语言进行微生物组数据分析。本讲将系统介绍微生物组数据分析的原理、方法和实践操作，帮助您掌握从原始测序数据到生物学解释的完整分析流程。

微生物组研究概述

微生物组（Microbiome）是指特定环境中所有微生物及其遗传物质的总和，包括细菌、古菌、真菌、病毒等。随着高通量测序技术的发展，16S rRNA 基因扩增子测序已成为研究微生物群落组成和多样性的主流方法。通过分析微生物组数据，我们可以：

- 揭示群落组成：了解样本中存在的微生物种类及其相对丰度
- 评估群落多样性：从 Alpha 多样性（样本内）和 Beta 多样性（样本间）两个维度刻画群落特征
- 识别生物标志物：发现与特定环境条件或宿主状态相关的关键微生物类群
- 解析共现网络：构建微生物间的相互作用网络，推断生态关系

分析流程概览

本讲涵盖微生物组数据分析的完整流程。下图展示了 `microeco` 包的整体分析框架：

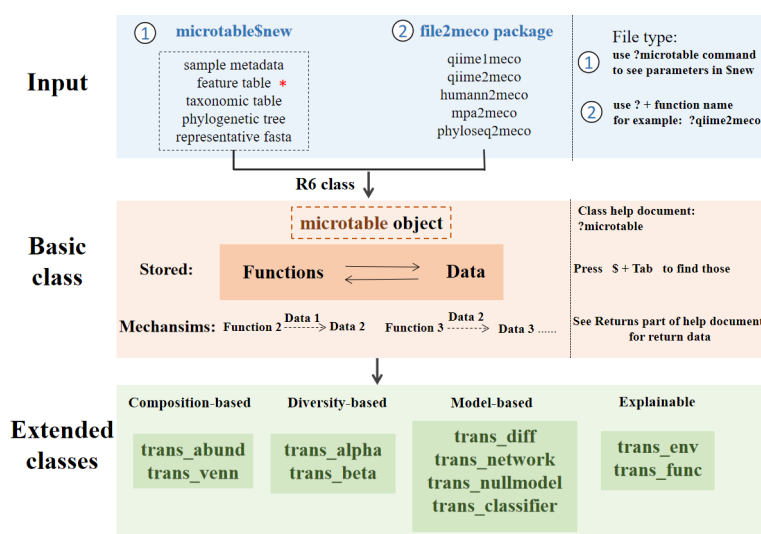


图 1: `microeco` 包的分析框架

本讲内容组织

1. 测序数据预处理（[第一章](#)）：使用 DADA2 对原始测序数据进行质量控制、去噪和 ASV（Amplicon Sequence Variants）推断
2. 构建 `microtable` 对象（[第二章](#)）：将数据整理为 `microeco` 包所需的格式，建立标准化分析基础
3. 群落结构分析（[第三章](#)）：可视化物种组成，了解群落的基本特征
4. Alpha 多样性分析（[第四章](#)）：评估样本内的物种丰富度和多样性
5. Beta 多样性分析（[第五章](#)）：比较不同样本间的群落差异
6. 差异分析（[第六章](#)）：鉴定在不同处理组间显著差异的微生物类群
7. 网络分析（[第七章](#)）：构建微生物共现网络，解析物种间的相互作用

示例数据说明

本讲使用一组土壤微生物组数据作为示例。该数据来自栗河（LiHe）地区的土壤样本，包含三种不同处理组（ZY0、ZY0.5、ZY1），每组 4 个生物学重复，共计 12 个样本。测序 targeting 16S rRNA 基因的 V4 区，使用 Illumina MiSeq 平台进行双端测序（2×250 bp）。同时，我们还收集了每个样本的环境因子数据，包括 pH、电导率（EC）、总有机碳（TOC）和总有机质（TOM）。

软件环境

本讲主要使用以下 R 包进行数据分析：

- **dada2**：用于原始测序数据的预处理和质量控制
- **microeco**：用于微生物组数据的全面分析和可视化
- **tidyverse**：用于数据整理和可视化

请确保在运行本讲代码前已安装并加载了这些包。同时，本讲使用 **renv** 包管理 R 环境依赖，以确保分析的可重复性。

核心概念速览

在开始之前，让我们快速了解本讲涉及的几个核心概念：

ASV 与 OTU

- **ASV (Amplicon Sequence Variants, 扩增子序列变体)**：基于精确序列差异（单核苷酸分辨率）定义的微生物单元。DADA2 等工具通过错误率校正将相似序列聚合成 ASV，能够区分仅相差一个碱基的序列。
- **OTU (Operational Taxonomic Unit, 操作分类单元)**：传统的聚类方法，通常基于 97% 序列相似度将序列分组。OTU 可能掩盖真实的生物学变异。

为何选择 ASV？ASV 提供了更高的分辨率，能够检测细微的序列差异，且结果在不同研究间更具可比性（ASV 是“精确的标签”，而非依赖于特定聚类阈值）。

microtable 对象

microtable 是 **microeco** 包的核心数据结构，类似于 **phyloseq** 包的 **phyloseq** 对象。它将微生物组分析的三大核心数据表整合在一起：

- **OTU/ASV 表**：特征（微生物）× 样本的丰度矩阵
- **样本表**：样本的元数据（分组、环境因子等）
- **分类表**：特征的分类学注释（界/门/纲/目/科/属/种）

构建 **microtable** 对象是所有后续分析的基础，本书将在第二章详细讲解其构建方法。

组成性效应 (Compositional Effect)

微生物组数据是组成性数据——每个样本的序列总数（测序深度）是固定的，因此一个分类群的相对丰度增加必然意味着其他分类群的相对丰度减少。这会导致“虚假相关”现象：

- 两个分类群可能在统计上负相关，但这只是因为第三个分类群丰度增加
 - 差异分析时需要特别注意，某分类群的“富集”可能是绝对丰度增加，也可能只是其他分类群减少
- 本书将在差异分析和网络分析章节讨论如何缓解组成性效应的影响。

使用说明

本书中的所有代码均可直接在 R 环境中运行。建议读者在阅读时同步执行代码，以加深理解。

💡 环境准备

从课程仓库下载本课程的源代码：


```
git clone git@github.com:D2RS-2026spring/lecture6-r-microbiome-analysis.git
```

在开始学习之前，请确保你的 R 版本 $\geq 4.1.0$ ，并已安装以下软件包：

```
cat(paste0(" 本次使用的 R 版本: ", R.version.string))
```

本次使用的 R 版本：R version 4.5.2 (2025-10-31)

```
install.packages("pak") # 推荐使用 pak 包安装
pak::pak(c("tidyverse", "microeco", "dada2", "ggplot2",
           "cowplot", "pheatmap", "vegan", "randomForest"))
```

如果你了解 renv，还可以一键创建一个可重复的本地分析环境。

```
renv::restore()
```

然后在终端运行 `quarto render --to html` 编译全书内容¹。

学习建议

- 每个章节都按照“概述-流程-实现-结果”的结构组织，建议按顺序阅读
 - 代码块可以直接运行，但也鼓励您根据实际数据进行调整
 - 理解分析背后的生物学意义比单纯运行代码更重要
 - 注意记录分析过程中的参数设置和关键决策，以保证研究的可重复性
- 希望本讲能帮助您建立系统的微生物组数据分析思维，祝学习愉快！

¹本项目推荐使用 [Positron](#) 作为开发环境。用 Positron 打开本项目后，renv 会自动识别并恢复依赖环境，Quarto 文档也可以直接预览和渲染。如果运行 `quarto render` 将同时生成 HTML 和 PDF。因 PDF 渲染配置容易出问题，所以如果出错，推荐先生成 HTML 文档。

第 1 章 测序数据预处理

1.1 数据准备

在开始分析之前，请确保已下载原始测序数据。

⚠ 检查数据文件

以下代码将检查数据文件是否存在：

```
# 检查原始测序数据目录
PATH_SEQENCING_RAW <- "data/amplicon/raw/"
data_exists <- dir.exists(PATH_SEQENCING_RAW) && length(list.files(PATH_SEQENCING_RAW, pattern = "\
data_exists

#> [1] TRUE
```

如果 data_exists 返回 FALSE，说明数据文件尚未下载。请按以下步骤获取数据：

1. 下载数据文件：
 - 地址：<https://cloud.bio-spring.top/index.php/s/CZqYgzJWLczo2Rp>
 - 备用：QQ 群文件 lecture6-amplicon-rawdata.zip
2. 解压到项目目录：

```
unzip data.zip -d /path/to/lecture6-r-microbiome-analysis/
```

3. 验证目录结构：

```
data/
├─ amplicon/
│   └─ raw/           # 原始测序 fastq 文件
├─ processed/        # 预处理后的数据（该文件随本仓库提供，不在压缩包中）
└─ silva/             # 参考数据库
```

1.2 分析概述

采用 DADA2 对公司返回的数据进行分析注释。

1.3 加载需要的包

```
library(dada2)
library(ggplot2)
library(cowplot)
library(dplyr)
library(stringr)
library(tibble)
```

1.4 检查实验数据

这些 fastq 文件是通过 2x250 Illumina Miseq 扩增子对 16S rRNA 基因的 V4 区进行测序生成的，这些样本来自实验。现在，只需将它们视为要处理的配对端 fastq 文件。

```
# 设置文件所在的路径
PATH_SEQENCING_RAW <- "data/amplicon/raw/"

# 查找并排序 R1 和 R2 文件
```

```
fnFs <- sort(list.files(PATH_SEQENCING_RAW, pattern="_R1.f.*q.gz", full.names = TRUE))
fnRs <- sort(list.files(PATH_SEQENCING_RAW, pattern="_R2.f.*q.gz", full.names = TRUE))

c(fnFs, fnRs) |> basename() |> sort()
```

```
#> [1] "ZTPSN25UE442-Sep25LHK0_1_R1.fq.gz" "ZTPSN25UE442-Sep25LHK0_1_R2.fq.gz"
#> [3] "ZTPSN25UE443-Sep25LHK0_2_R1.fq.gz" "ZTPSN25UE443-Sep25LHK0_2_R2.fq.gz"
#> [5] "ZTPSN25UE444-Sep25LHK0_3_R1.fq.gz" "ZTPSN25UE444-Sep25LHK0_3_R2.fq.gz"
#> [7] "ZTPSN25UE450-Sep25LHK1_1_R1.fq.gz" "ZTPSN25UE450-Sep25LHK1_1_R2.fq.gz"
#> [9] "ZTPSN25UE451-Sep25LHK1_2_R1.fq.gz" "ZTPSN25UE451-Sep25LHK1_2_R2.fq.gz"
#> [11] "ZTPSN25UE452-Sep25LHK1_3_R1.fq.gz" "ZTPSN25UE452-Sep25LHK1_3_R2.fq.gz"
```

1.4.1 生成 metadata

文件名包含 Sample ID，这里直接从 sample ID 获取实验处理。

```
sample.names = fnFs |>
  basename() |>
  str_extract("Sep25.*_[0-9]") |>
  str_remove("^Sep25")

meta = tibble(sample_id = sample.names) |>
  tidyr::separate(sample_id, c("treatment", "rep"), sep = "_", remove = FALSE)
```

1.4.2 查看文件的测序质量

测序质量的分布是一个重要的指标，通常会查看前 3 个样本的测序质量图 1.1。由图可知：

- R1 的测序治疗较 R2 差，R2 的测序质量平均值都在 30 以上，且没有明显的波动；
- 在 225 bp 以后，序列的整体长度会有明显的下降。

```
p1 = plotQualityProfile(fnFs[1:3]) +
  facet_wrap(~file, ncol=1)

p2 = plotQualityProfile(fnRs[1:3]) +
  facet_wrap(~file, ncol=1) +
  ylab("")

plot_grid(p1, p2, ncol=2)
```

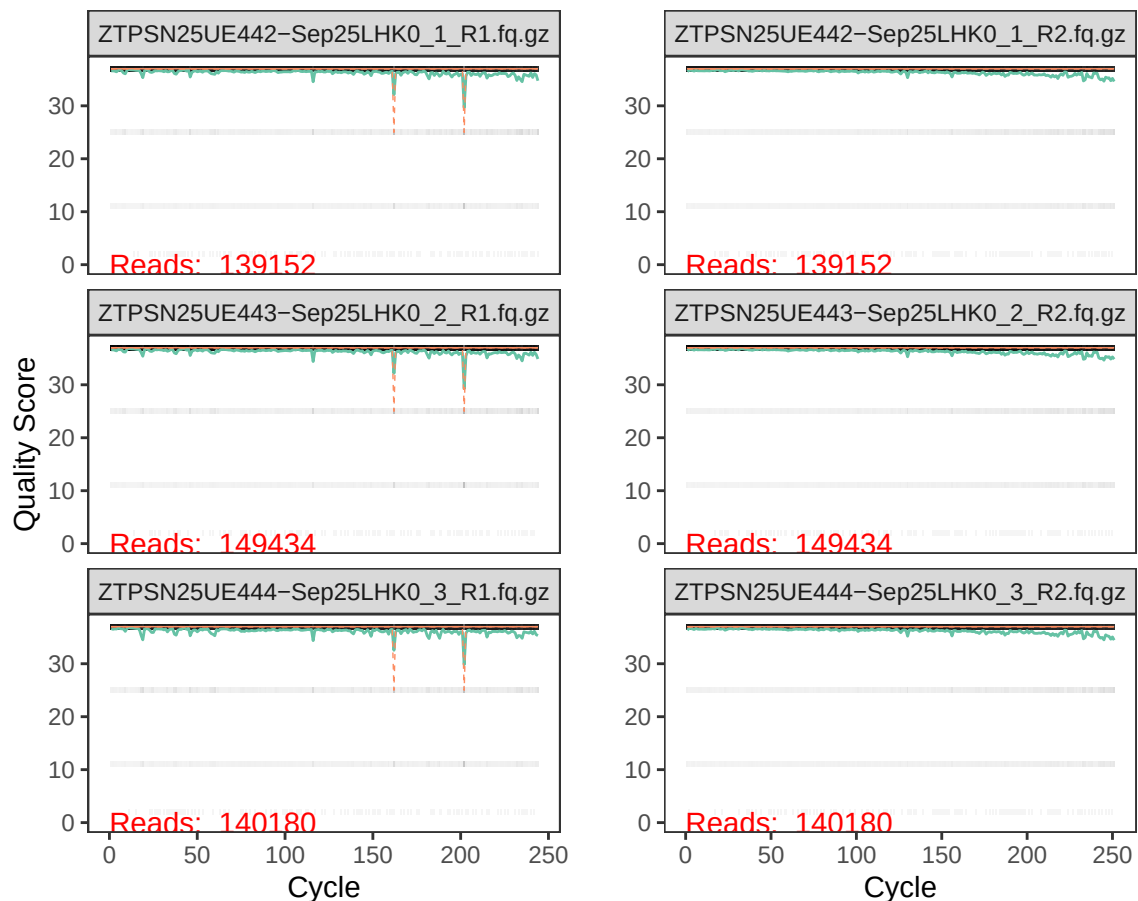


图 1.1: 前 3 个的测序质量

测序质量得分在每个位置的分布以灰度热图显示，其中深色对应更高的频率。绘制的线条显示位置统计数据：绿色为平均值，橙色为中位数，虚线橙色为第 25 和第 75 百分位数。如果序列长度不同，将绘制一条红线，显示至少延伸到该位置的读数百分比。

1.5 过滤和修剪

1.5.1 设置过滤文件的产生路径和名称

```
# Place filtered files in filtered/ subdirectory
filtFs <- file.path(PATH_SEQENCING_RAW, "filtered", paste0(sample.names, "_F_filt.fq.gz"))
filtRs <- file.path(PATH_SEQENCING_RAW, "filtered", paste0(sample.names, "_R_filt.fq.gz"))
names(filtFs) <- sample.names
names(filtRs) <- sample.names
```

1.5.2 对原始数据进行过滤

```
out <- filterAndTrim(fnFs, filtFs, fnRs, filtRs,
  maxN = 0, maxEE = c(2,4), truncQ = 0,
  rm.phix = FALSE,
  truncLen = c(200,200),
  compress = TRUE,
  multithread = TRUE)
head(out)
```

```
#>
#> ZTPSN25UE442-Sep25LHK0_1_R1.fq.gz 139152 134592
```

```
#> ZTPSN25UE443-Sep25LHK0_2_R1.fq.gz 149434 144540
#> ZTPSN25UE444-Sep25LHK0_3_R1.fq.gz 140180 134535
#> ZTPSN25UE450-Sep25LHK1_1_R1.fq.gz 128633 124752
#> ZTPSN25UE451-Sep25LHK1_2_R1.fq.gz 133500 128168
#> ZTPSN25UE452-Sep25LHK1_3_R1.fq.gz 143316 137417
```

1.5.3 估计测序错误率

在 DADA2 中, `learnErrors` 是整个流程的关键步骤之一,其目的是学习测序错误模型,这是 DADA2 高精度去噪能力的基础。

高通量测序(如 Illumina)在读取 DNA 序列时会出现错误,比如碱基错配(substitution)。DADA2 的目标是区分这些测序错误与真实的生物变异(例如不同的细菌或物种)。

为了做到这一点, DADA2 需要知道测序错误是如何分布的,例如:

- 一个“A”被错误读成“T”的概率有多大?
- 在不同的测序质量分数下,错误概率如何变化?

`learnErrors(filtFs)` 会根据过滤后的序列数据(如 `filtFs`)统计这些碱基替换的发生频率,结合质量分数,拟合出一个错误率矩阵(error rate matrix),这个矩阵是后续序列去噪(denoising)的基础。

只有有了这个模型, DADA2 才能:

1. 判断某条序列的观察频次是否能用测序错误解释;
2. 决定是否要把这条序列当作真实变体(ASV)保留下来。

对同一台机器、同一批次的样本,可以用一个共享的错误模型。如果你有很多样本,也可以选择只在一部分样本中学习错误,再应用到其他样本中。

学习正向和反向的错误模型

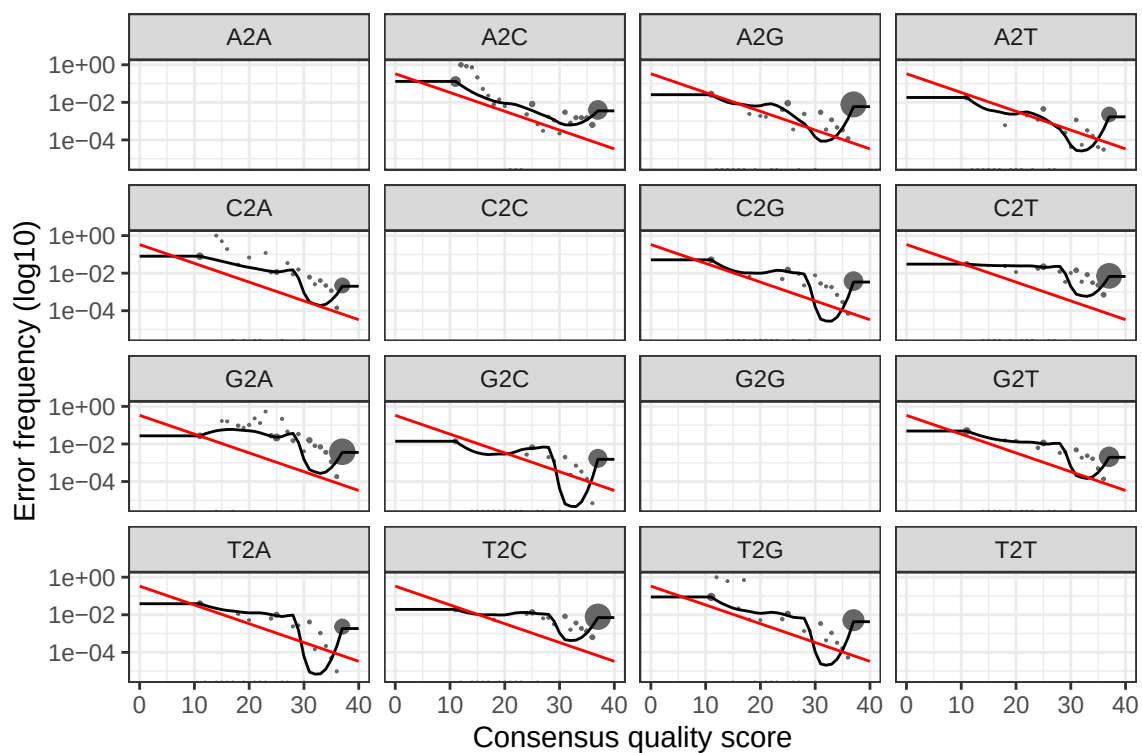
```
errF <- learnErrors(filtFs, multithread = TRUE)
```

```
#> 107683800 total bases in 538419 reads from 4 samples will be used for learning the error rates.
```

```
errR <- learnErrors(filtRs, multithread = TRUE)
```

```
#> 107683800 total bases in 538419 reads from 4 samples will be used for learning the error rates.
```

```
plotErrors(errF, nominalQ = TRUE)
```



(a) 红色线是理论上的错误率，黑色点是观测错误率，黑色线是拟合（估计）的错误率

图 1.2: 正向错误模型

```
plotErrors(errR, nominalQ = TRUE)
```

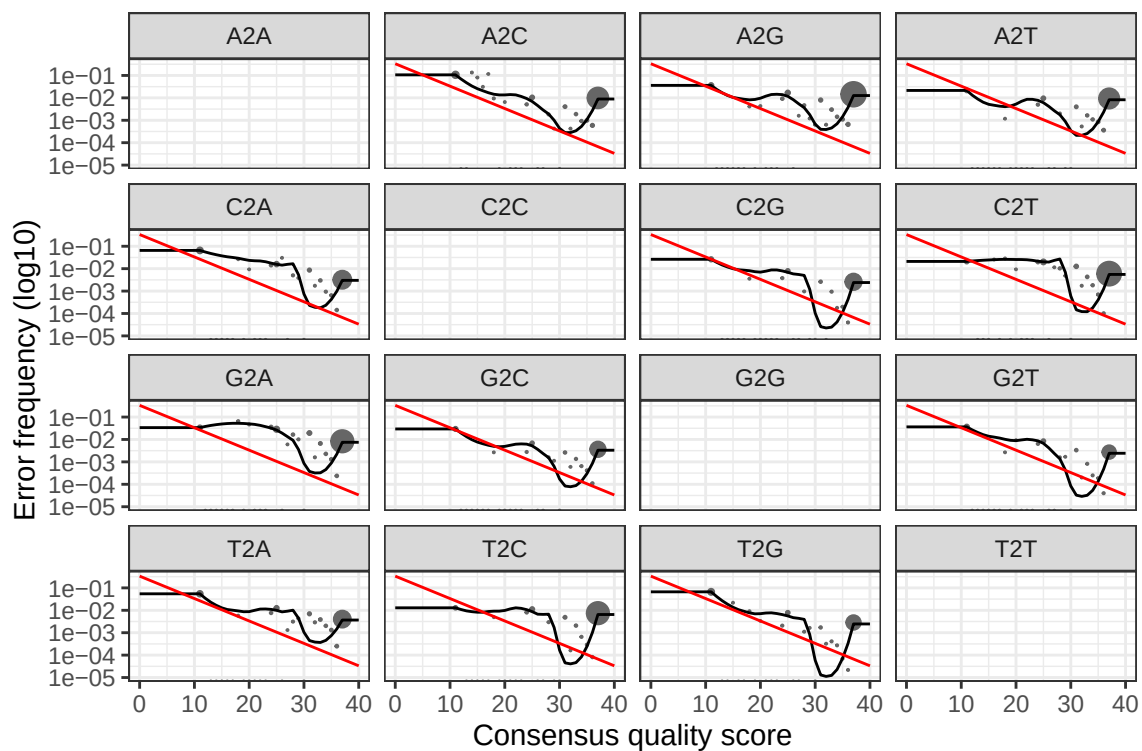


图 1.3: 反向错误模型

1.5.4 序列去重

```
# 去除重复序列
# 仅仅保留重复序列中的一条序列，以在后续序列处理中节省计算资源
derepFs <- derepFastq(filtFs)
derepRs <- derepFastq(filtRs)
```

1.5.5 根据模型进行匹配

```
dadaFs <- dada(filtFs, err=errF, multithread = TRUE, verbose = FALSE)
dadaRs <- dada(filtRs, err=errR, multithread = TRUE, verbose = FALSE)
```

```
dadaFs[[1]]
```

```
#> dada-class: object describing DADA2 denoising results
#> 1838 sequence variants were inferred from 45002 input unique sequences.
#> Key parameters: OMEGA_A = 1e-40, OMEGA_C = 1e-40, BAND_SIZE = 16
```

1.6 ASV 表

1.6.1 序列拼接

```
mergers2 <- mergePairs(dadaFs, derepFs, dadaRs, derepRs)
```

下面，查看合并后的结果。

- \$abundance: 对应这个正向/反向序列组合的读数数量。
- \$sequence: 合并后的序列。
- \$forward: 正向去噪序列的索引。
- \$reverse: 反向去噪序列的索引。
- \$nmatch: 重叠区域中匹配的核苷酸数量。
- \$nmismatch: 重叠区域中不匹配的核苷酸数量。
- \$nindel: 重叠区域中的插入/缺失数量。
- \$prefer: 用于重叠区域的序列来源。1= 正向；2= 反向。
- \$accept: 如果正向和反向去噪序列之间的重叠至少为 minOverlap 且最多有 maxMismatch 差异，则为 TRUE。否则为 FALSE。

```
# 查看合并后的序列
```

```
head(mergers2[[1]]) |> as_tibble()
```

```
#> # A tibble: 6 x 9
#>   sequence      abundance forward reverse nmatch nmismatch nindel prefer accept
#>   <chr>          <int>    <int>   <int>   <int>    <int>   <int>   <dbl> <lgl>
#> 1 GTGTCAGCCGCCG~    159     252     434    108         0         0         1 TRUE
#> 2 GTGTCAGCAGCCG~    142         1         1    108         0         0         1 TRUE
#> 3 GTGTCAGCAGCCG~    125     469    1046    108         0         0         1 TRUE
#> 4 GTGTCAGCCGCCG~    121         2         1    108         0         0         1 TRUE
#> 5 GTGCCAGCAGCCG~    113         3         1    108         0         0         1 TRUE
#> 6 GTGCCAGCCGCCG~    108         4         1    108         0         0         1 TRUE
```

1.6.2 生成 ASV 表

```
# 生成 ASV 表
seqtab <- makeSequenceTable(mergers2)
dim(seqtab)
```

```
#> [1]      6 22183
```


1.6.3 选择目标长度的序列

515F 和 **806R** 理论上扩增区域 约 **253 bp**（不含引物序列）或者 **293 bp**（含引物序列）。

- 515F: GTGYCAGCMGCCGCGGTAA
- 806R: GGACTACNVGGGTWTCTAAT

使用 515F–806R 进行 16S V4 区扩增，拼接后得到的序列是完全正常、符合设计预期的图 1.4。

```
# 查看序列长度分布
```

```
table(nchar(getSequences(seqtab))) |> plot(type = 'l')
```

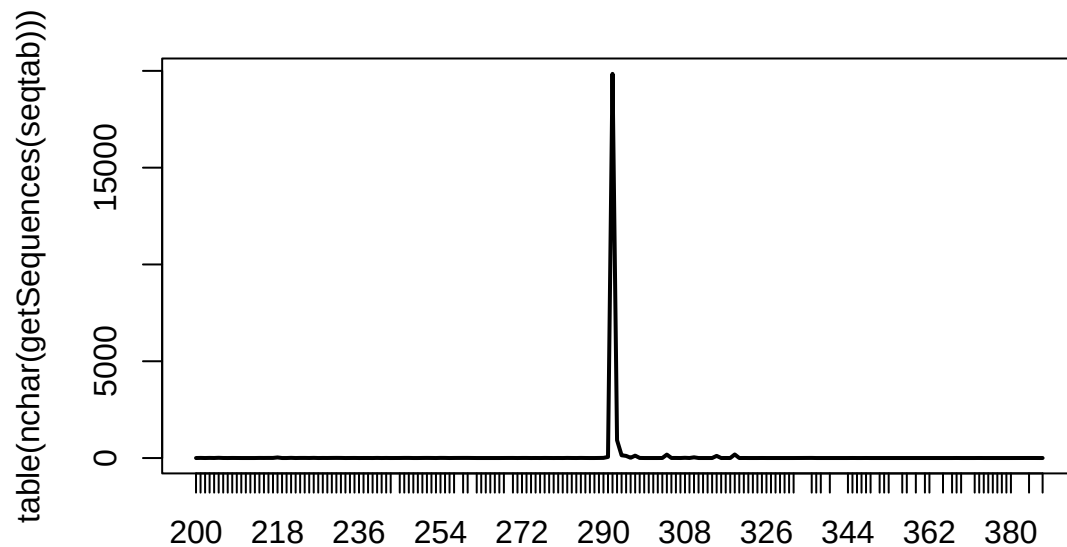


图 1.4: 拼接后 ASV 序列的长度分布

```
# 筛选目标长度的序列
```

```
seqtab2 <- seqtab[, nchar(getSequences(seqtab)) %in% 290:295]
```

```
# 检查筛选后的维度
```

```
dim(seqtab2)
```

```
#> [1] 6 21083
```

1.6.4 去嵌合体

```
# 去除嵌合体
```

```
seqtab.nochim <- removeBimeraDenovo(seqtab2, method="pooled", multithread=F, verbose=TRUE)
```

```
# 统计去除嵌合体后的 ASV 数量
```

```
dim(seqtab.nochim)
```

```
#> [1] 6 7125
```

```
# 统计去除嵌合体前后的 ASV 数量占比
```

```
sum(seqtab.nochim)/sum(seqtab)
```

```
#> [1] 0.5474293
```

1.6.5 测序数据处理情况追踪

```
# 求和函数
getN <- function(x) sum(getUniques(x))

# 合并各样本分步数据量
track <- cbind(out, sapply(dadaFs, getN), sapply(dadaRs, getN), sapply(mergers2, getN), rowSums(seqtab.n
# If processing a single sample, remove the sapply calls: e.g. replace sapply(dadaFs, getN) with getN(da
# 修改列名
colnames(track) <- c("input", "filtered", "denoisedF", "denoisedR", "merged", "nonchim")
# 行名修改为样本名
rownames(track) <- sample.names
# 统计结果预览
head(track)

#>           input filtered denoisedF denoisedR merged nonchim
#> LHK0_1 139152   134592   129009   126126  97182   54164
#> LHK0_2 149434   144540   137150   133207  98621   66147
#> LHK0_3 140180   134535   129277   126297 100141   54221
#> LHK1_1 128633   124752   121857   120977 108376   56047
#> LHK1_2 133500   128168   124265   121607 100480   51746
#> LHK1_3 143316   137417   132756   131321 107880   53074
```

1.7 分配分类

1.7.1 分类注释

```
SILVA_DB_FILE = "data/silva/silva_nr99_v138.2_wSpecies_train_set.fa.gz"

# 分配到属级别
taxa <- assignTaxonomy(seqtab.nochim, SILVA_DB_FILE, multithread = TRUE, minBoot = 60)

# 打印结果
taxa |> as_tibble()

#> # A tibble: 7,125 x 7
#>   Kingdom Phylum      Class      Order      Family Genus Species
#>   <chr>    <chr>      <chr>      <chr>      <chr>  <chr> <chr>
#> 1 Bacteria Gemmatimonadota Gemmatimonadia Gemmatimonadales Gemma~ <NA> <NA>
#> 2 Bacteria Gemmatimonadota Gemmatimonadia Gemmatimonadales Gemma~ <NA> <NA>
#> 3 Bacteria Gemmatimonadota Gemmatimonadia Gemmatimonadales Gemma~ <NA> <NA>
#> 4 Bacteria Gemmatimonadota Gemmatimonadia Gemmatimonadales Gemma~ <NA> <NA>
#> 5 Bacteria Gemmatimonadota Gemmatimonadia Gemmatimonadales Gemma~ <NA> <NA>
#> 6 Bacteria Gemmatimonadota Gemmatimonadia Gemmatimonadales Gemma~ <NA> <NA>
#> 7 Bacteria Gemmatimonadota Gemmatimonadia Gemmatimonadales Gemma~ <NA> <NA>
#> 8 Bacteria Gemmatimonadota Gemmatimonadia Gemmatimonadales Gemma~ <NA> <NA>
#> 9 Bacteria Gemmatimonadota Gemmatimonadia Gemmatimonadales Gemma~ <NA> <NA>
#> 10 Bacteria Gemmatimonadota Gemmatimonadia Gemmatimonadales Gemma~ <NA> <NA>
#> # i 7,115 more rows
```

1.8 保存结果

1.8.1 保存 ASV 序列

将 ASV 序列保存为 FASTA 格式，以便后续使用。

```
asv_sequence <- getSequences(seqtab.nochim)
```

为序列指定唯一 ID。

```
library(digest)
get_dna_id = function(seq, length = 12){
  hash = digest(seq, algo = "sha256", serialize = FALSE)
  substr(hash, 1, length)
}
```

```
# 生成 ASV ID
```

```
asv_id = paste0("ASV", sapply(asv_sequence, get_dna_id))
```

```
# 将数据框转换为 FASTA 格式
```

```
asv_fasta = paste0(">", asv_id, "\n", asv_sequence)
```

```
# 保存为 FASTA 文件
```

```
writeLines(asv_fasta, "data/amplicon/sep25-asv.fasta")
```

1.8.2 保存 ASV 表和分类表

将 seqtab.nochim 和 taxa 保存为 CSV 文件，便于后续分析和共享。

```
# ASV 表格导出
```

```
# 这里的 seqtab.nochim 是一个数值矩阵，行名是样本名称，列名是 ASV 序列
```

```
colnames(seqtab.nochim) <- asv_id
```

```
write.csv(seqtab.nochim, "data/amplicon/sep25-asvtab.csv")
```

```
# 分类表格导出
```

```
# 这里的 taxa 是一个文本矩阵，行名是 ASV 序列，列名是分类学层级
```

```
rownames(taxa) <- asv_id
```

```
write.csv(taxa, "data/amplicon/sep25-taxtab.csv")
```


第 2 章 群落结构分析

2.1 概述

在前一章中，我们使用 DADA2 对原始测序数据进行了预处理，得到了 ASV 表、分类表和样本表。本章将基于这些数据进行群落结构分析——微生物组数据分析的第一步。

2.1.1 什么是群落结构分析？

群落结构分析旨在回答两个基本问题：

1. “样本中存在哪些微生物？”——识别群落的成员组成
2. “它们的相对丰度如何？”——了解各成员在群落中的占比

通过可视化物种组成，我们可以：

- 了解群落概貌：识别优势菌群，把握群落的基本特征
- 发现组间差异：比较不同处理组间的组成差异，形成初步假设
- 指导后续分析：为差异分析、网络分析等提供线索

2.1.2 为何群落结构分析是第一步？

在微生物组数据分析的标准流程中，群落结构分析通常作为探索性数据分析（EDA）的起点：

1. 数据质量检查：通过物种组成图可以直观发现异常样本（如某样本组成与其他重复差异极大，可能存在污染）
2. 初步假设生成：肉眼观察到的组间差异可以指导后续的统计检验方向
3. 分析可行性评估：判断样本分组是否合理，是否需要调整分组策略

2.1.3 microtable 对象：分析的基础

本章所有分析都基于 microtable 对象。下图展示了 microtable 的内部结构和关键函数：

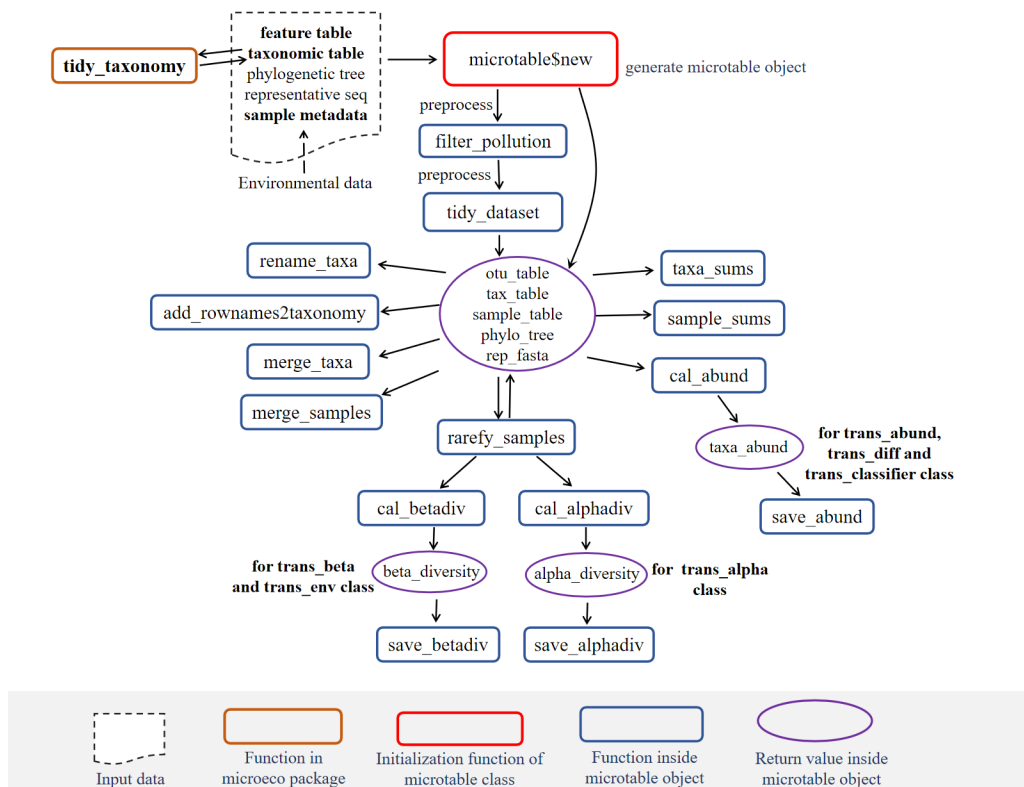


图 2.1: microtable 对象结构框架

图注：矩形框内为函数，红色矩形表示重要函数，虚线框表示需要关注的关键对象

microtable 对象整合了三大核心数据表：

- **otu_table**: ASV × 样本的丰度矩阵
- **sample_table**: 样本的元数据（如处理组、环境因子）
- **tax_table**: ASV 的分类学注释

本章将通过 trans_abund 类对 microtable 对象进行转化和可视化。

2.1.4 分析内容

物种组成分析通常在多个分类等级（门、纲、目、科、属）进行，以不同分辨率展示群落结构。常用的可视化方式包括：

图表类型	适用场景	优势
堆叠柱状图	展示各样本的物种组成比例	直观比较组间差异
热图	展示高丰度物种在各样本中的分布	适合大量分类群，可聚类
箱线图	比较不同组间某分类群的丰度分布	显示组内变异和统计差异
饼图	展示组平均组成的整体比例	适合展示整体构成

2.2 流程

在 microeco 包中，群落结构分析主要通过 trans_abund 类实现，其工作流程如下：

1. 数据准备：确保 microtable 对象已正确构建，包含 OTU/ASV 表、样本表和分类表
2. 创建分析对象：使用 trans_abund\$new() 初始化分析对象，指定目标分类等级（如 Phylum、Genus）
3. 可视化：调用相应的绘图函数生成各种图表

trans_abund 类支持多种图表类型：

图表类型	函数	适用场景
堆叠柱状图	plot_bar()	展示各样本的物种组成比例
热图	plot_heatmap()	展示高丰度物种在各样本中的分布
箱线图	plot_box()	比较不同组间某分类群的丰度分布
饼图	plot_pie()	展示组平均组成的整体比例

2.3 实现

2.4 构建 microtable 对象

这里使用 microeco 软件包进行数据分析。

首先需要从数据保存的目录读取数据，并构建 microtable 对象。

```
PATH = "data/amplicon/processed"
list.files(PATH, full.names = TRUE)

#> [1] "data/amplicon/processed/asv.fasta"
#> [2] "data/amplicon/processed/asvtab.csv"
#> [3] "data/amplicon/processed/metadata.csv"
#> [4] "data/amplicon/processed/taxtab.csv"

metadata_file = file.path(PATH, "metadata.csv")
otu_table_file = file.path(PATH, "asvtab.csv")
tax_table_file = file.path(PATH, "taxtab.csv")
sequence_file = file.path(PATH, "asv.fasta")
```

2.4.1 载入包

```
library(tidyverse)
library(microeco)

theme_set(theme_bw())
```

2.4.2 OTU 表

```
otu_table = read_csv(otu_table_file)
otu_table = column_to_rownames(otu_table, var = names(otu_table)[1])
```

在 *microeco* 包（以及大多数 R 语言生物信息包）的规范中，*microtable* 对 *otu_table* 的格式有严格要求：行必须是 **Taxa**（ASV/OTU），列必须是 **Samples**。

如果你的数据正好反过来了（行是样品，列是 ASV），需要在传入 *otu_table* 之前，使用 *t()* 函数对其进行转置，并确保它仍然是一个 *data.frame*。

```
otu_table <- as.data.frame(t(otu_table))
```

为了让你直观理解 *microtable* 的结构逻辑，可以参考下图的对应关系：

- **OTU Table**: 行名必须对接 **Taxonomy Table** 的行名（即 ASV ID）。
- **OTU Table**: 列名必须对接 **Sample Table** 的行名（即 Sample ID）。

2.4.3 Sample 表

```
sample_table = read_csv(metadata_file)
sample_table = column_to_rownames(sample_table, var = names(sample_table)[1])
```

2.4.4 Taxonomy 表

```
taxon_table = read_csv(tax_table_file)
taxon_table = column_to_rownames(taxon_table, var = names(taxon_table)[1]) |>
  mutate(Genus = if_else(str_detect(Genus, "Burkholderia"), "Burkholderia", Genus))
```

2.4.5 构建对象

构建 *microtable* 对象。

```
mt = microtable$new(otu_table = otu_table, sample_table = sample_table, tax_table = taxon_table)
mt
```

```
#> microtable-class object:
```

```
#> sample_table have 12 rows and 8 columns
```

```
#> otu_table have 7230 rows and 12 columns
```

```
#> tax_table have 7230 rows and 7 columns
```

处理 Taxonomic 表格中的 NA 值。

- 删掉 *Phylum* == NA 的 ASV（根本没比对上的垃圾序列）。

```
mt$tax_table = subset(mt$tax_table, !is.na(Phylum))
mt$tidy_dataset()
```

- 为未知的 Genus 填充数据（自上级非 NA）。

```
# 该函数会自动查找分类表中的缺失值并填充
```

```
mt$tax_table <- tidy_taxonomy(mt$tax_table)
```

```
# 填充完成后，运行 tidy_dataset 同步一下
```

```
mt$tidy_dataset()
```

2.4.6 门水平的物种组成

首先，我们在门（Phylum）水平上分析物种组成。这有助于把握群落的整体框架，因为门水平的分类群数量相对较少，易于解释。

```
t1 = trans_abund$new(dataset = mt, taxrank = "Phylum", ntaxa = 8)
t1$plot_bar(xtext_angle = 45, facet = "treatment", xtext_keep = FALSE)
```

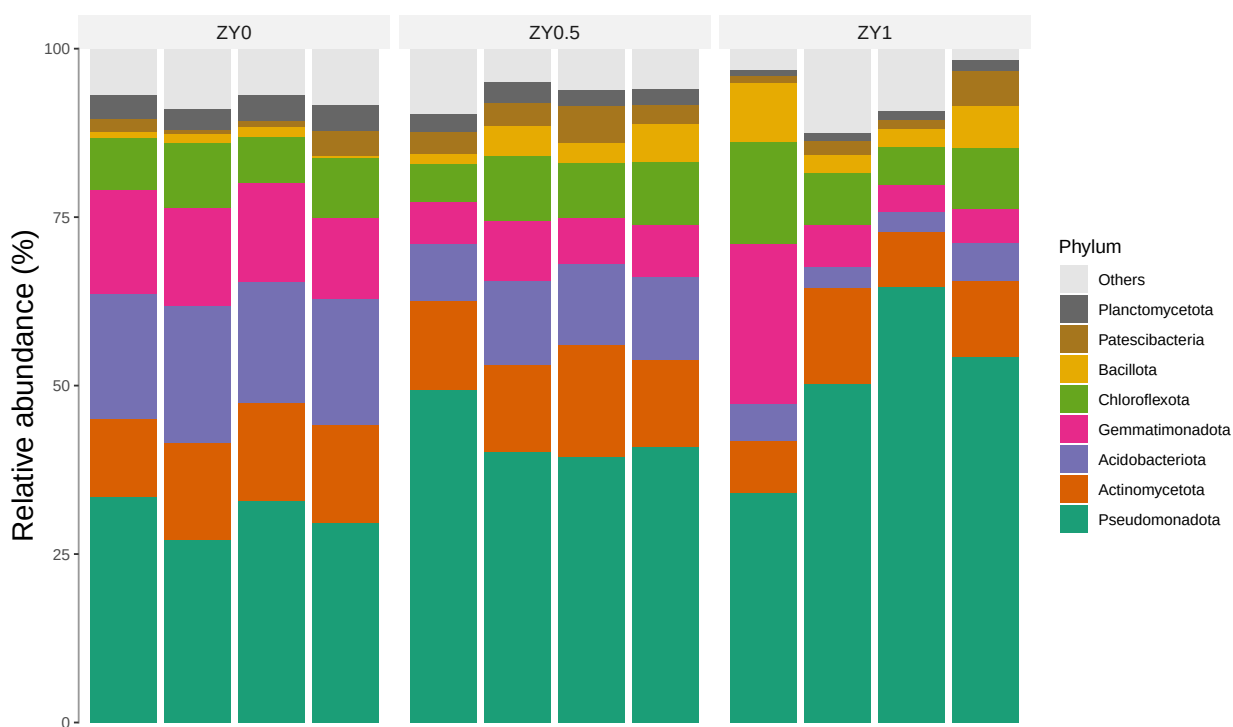


图 2.2: 门水平物种组成的堆叠柱状图

参数说明：

- `taxrank = "Phylum"`: 指定在门水平汇总数据
- `ntaxa = 8`: 选择丰度最高的 8 个门，其余归为“Others”
- `facet = "treatment"`: 按处理组分面展示
- `xtext_keep = FALSE`: 不显示样本名称，使图形更简洁

2.4.7 纲水平的物种组成

进一步在纲（Class）水平上分析，可以获得更精细的分类信息。

```
t1 = trans_abund$new(dataset = mt, taxrank = "Class", ntaxa = 15)
t1$plot_bar(xtext_angle = 45, facet = "treatment", xtext_keep = FALSE)
```

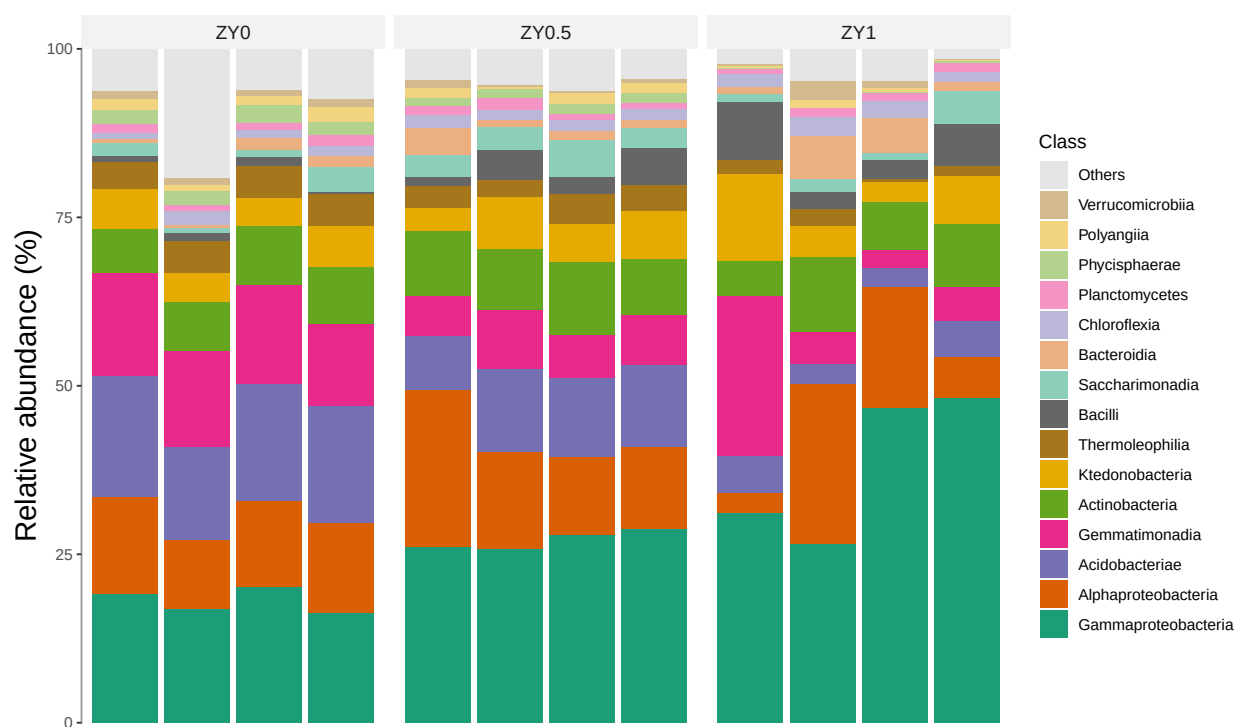



图 2.3: 纲水平物种组成的堆叠柱状图

2.4.8 属水平的热图

热图是展示大量分类群在多个样本中分布模式的有效方式。我们在属（Genus）水平上绘制热图，以观察高丰度属的分布特征。

```
t1 = trans_abund$new(dataset = mt, taxrank = "Genus", ntaxa = 30)
t1$plot_heatmap(heatmap = TRUE, facet = "treatment")
```

```
#> TableGrob (5 x 6) "layout": 6 grobs
#>   z      cells      name      grob
#> 1 1 (2-2,3-3) col_tree polyline[GRID.polyline.335]
#> 2 2 (4-4,1-1) row_tree polyline[GRID.polyline.336]
#> 3 3 (4-4,3-3) matrix    gTree[GRID.gTree.338]
#> 4 4 (5-5,3-3) col_names text[GRID.text.339]
#> 5 5 (4-4,4-4) row_names text[GRID.text.340]
#> 6 6 (3-5,5-5) legend    gTree[GRID.gTree.343]
```

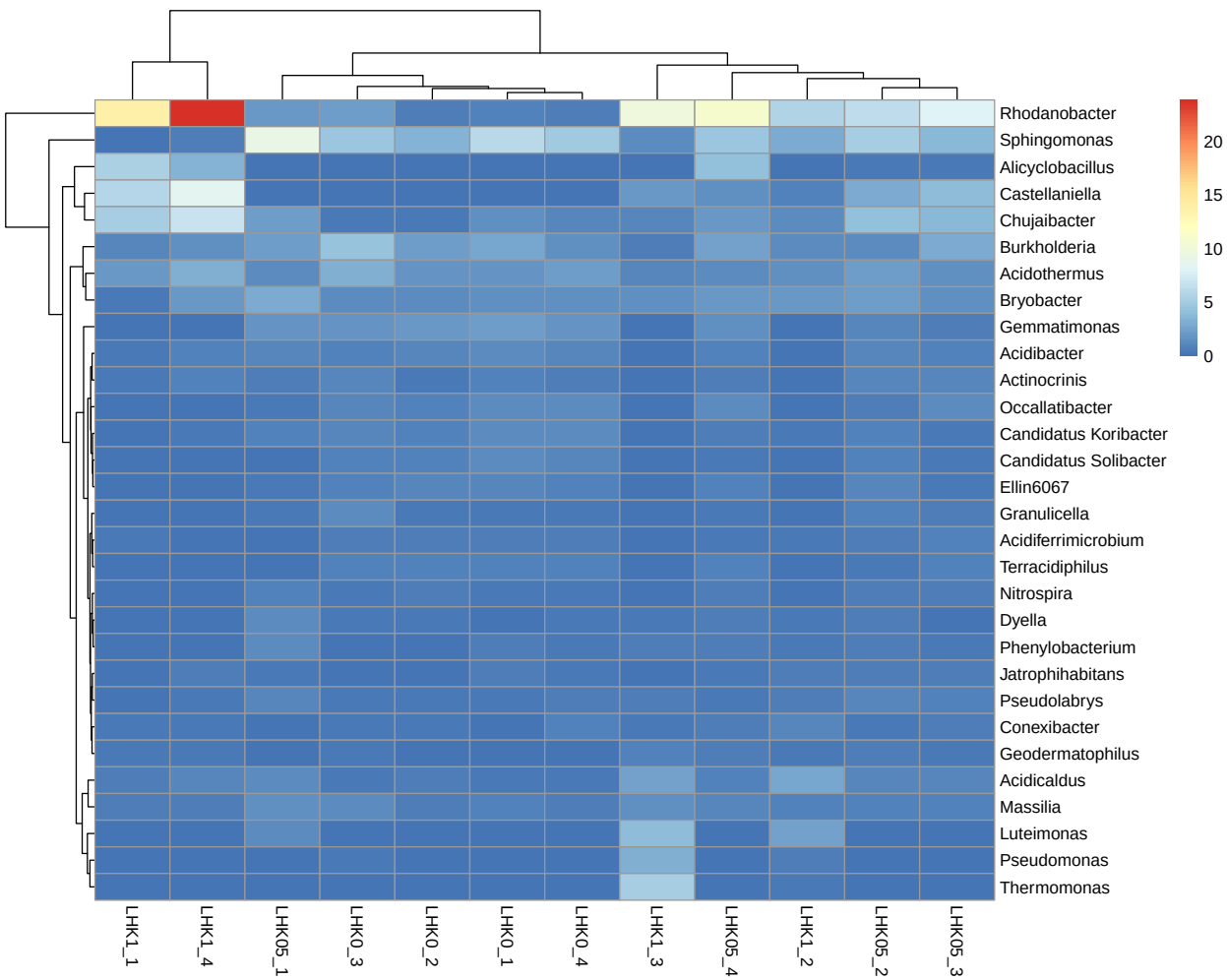


图 2.4: 属水平物种丰度热图

热图解读要点：

- 颜色深浅表示相对丰度的高低（通常颜色越深丰度越高）
- 行聚类显示分类群在样本间的相似分布模式
- 列聚类显示样本间的组成相似性

2.4.9 箱线图比较组间差异

箱线图可以直观地展示特定分类群在不同处理组间的丰度分布差异。

```
t1 = trans_abund$new(dataset = mt, taxrank = "Phylum", ntaxa = 5)
t1$plot_box(group = "treatment", xtext_angle = 0)
```

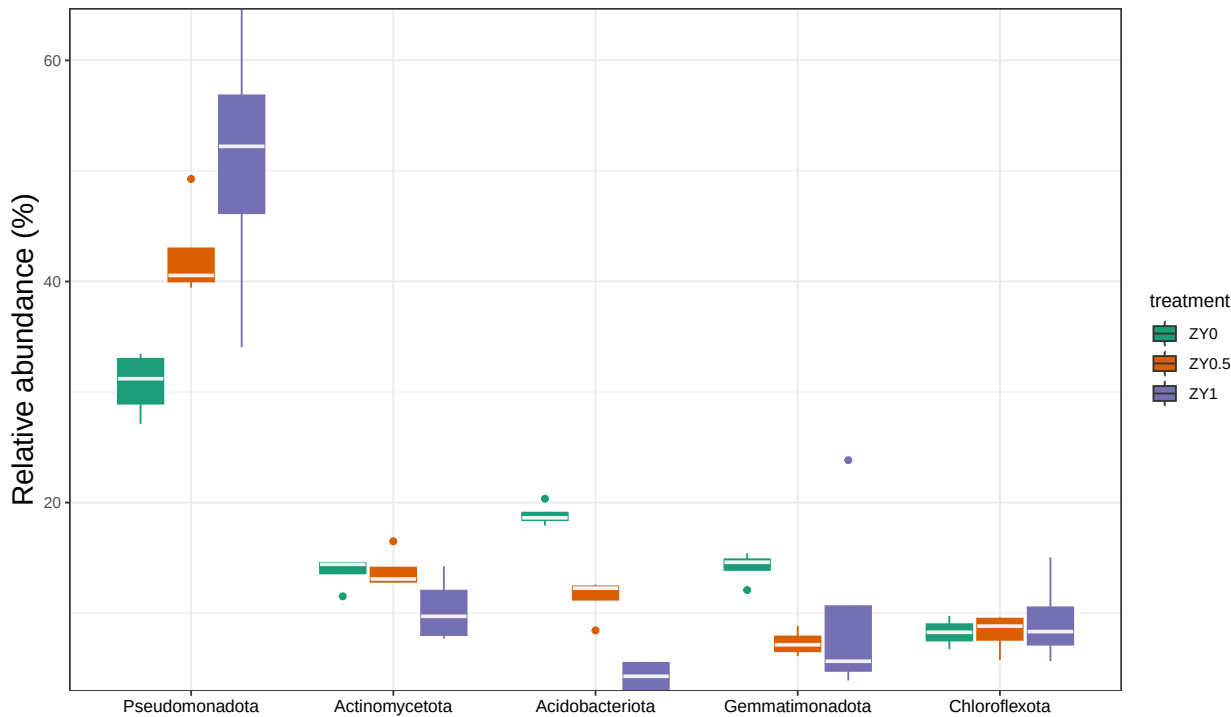


图 2.5: 门水平主要分类群的组间丰度分布

2.5 结果

2.5.1 群落组成特征

从门水平的分析结果（图 2.2）可以看出，本研究土壤微生物组主要由以下几个门组成：

- 变形菌门（**Proteobacteria**）：通常是最丰富的门之一，包含多种代谢类型的细菌
- 放线菌门（**Actinobacteria**）：在土壤中广泛分布，参与有机物分解
- 酸杆菌门（**Acidobacteria**）：土壤中的常见类群，适应酸性环境
- 芽孢杆菌门（**Bacillota**，原 **Firmicutes**）：包含多种芽孢形成菌

2.5.2 处理组间差异的初步观察

通过对比不同处理组（ZY0、ZY0.5、ZY1）的柱状图，我们可以初步观察到：

1. 优势门的相对比例变化：某些门在不同处理组间的相对丰度可能存在差异，这可能反映了处理条件对微生物群落的选择作用
2. 群落均匀度：各组内样本的重复性较好，表明组内变异相对较小

2.5.3 热图揭示的精细模式

属水平的热图（图 2.4）揭示了更精细的分布模式：

- 核心微生物组：某些属在所有样本中均有较高丰度，可能代表土壤微生物组的核心功能类群
- 处理组特异性：部分属在特定处理组中富集，这些类群可能是后续差异分析的重点关注对象

2.5.4 分析与讨论

群落结构分析的结果为理解处理效应提供了初步线索。在本案例中：

1. 环境适应性：不同处理条件（ZY0、ZY0.5、ZY1 可能代表不同施肥或处理强度）可能塑造了不同的微生物群落结构，反映了微生物对环境变化的适应
2. 功能推断：基于已知的分类群功能特征，我们可以初步推测：
 - 某些参与养分循环的门（如 **Proteobacteria** 和 **Actinobacteria**）的丰度变化可能与土壤肥力相关
 - 酸杆菌门的相对丰度可能与土壤 pH 条件有关
3. 后续分析方向：群落结构分析的结果可指导后续分析：

- 对于在柱状图中肉眼可见组间差异的分类群，可通过差异分析进行统计验证
- 热图中聚类在一起的分类群可能在功能上相关，可作为网络分析的候选对象

注意事项：物种组成分析展示的是相对丰度数据，在解释时需要注意组成性效应（Compositional effect），即一个分类群的相对丰度增加可能是由于其他分类群减少所致，而非其绝对丰度真正增加。

2.6 小提示

microeco 包的使用文档参见：https://chiliubio.github.io/microeco_tutorial。

- 使用 `tidy_taxonomy()` 函数确保分类表格式统一，特别是处理未知分类等级时
- 绘制热图时，`pheatmap = TRUE` 参数可以生成更美观的聚类热图
- 箱线图中添加 `add_sig = TRUE` 参数可以显示统计显著性标记（需先运行差异分析）

第 3 章 α 多样性

3.1 概述

Alpha 多样性 (α diversity) 用于描述单个样本内微生物群落的物种丰富度和均匀度，是微生物组分析中最基本的指标之一。理解 Alpha 多样性有助于评估：

- 群落复杂度：样本中包含多少种微生物？分布是否均匀？
- 生态位容纳能力：环境能支持多少物种共存？
- 干扰响应：环境变化如何影响群落内部的多样性？

3.1.1 常用 Alpha 多样性指标

指标	含义	计算方式	敏感性
Observed	观测到的 ASV/OTU 数量	直接计数	受测序深度影响
Chao1	估计物种总数	基于稀有物种推算	偏向稀有物种
Shannon	综合丰富度和均匀度	$H' = -\sum p_i \ln(p_i)$	中等敏感
Simpson	优势度指数	$\lambda = \sum p_i^2$	偏向优势种
Pielou	均匀度指数	$J' = H' / \ln(S)$	纯均匀度量

其中， p_i 是第 i 个物种的相对丰度， S 是物种总数。

3.1.2 分析目标

本分析旨在：

1. 评估不同处理组样本的 Alpha 多样性水平
2. 比较处理组间 Alpha 多样性的差异是否显著
3. 探讨环境因素与 Alpha 多样性的关系

3.2 流程

在 `microeco` 包中，Alpha 多样性分析的工作流程如下：

1. 数据标准化（抽平）：使用 `rarefy_samples()` 将各样本抽平到相同序列数，消除测序深度差异的影响
2. 计算多样性指标：使用 `cal_alphadiv()` 计算各类 Alpha 多样性指数
3. 差异检验：使用 `trans_alpha` 类进行组间差异统计检验
4. 可视化：绘制箱线图或小提琴图展示结果

关键注意事项：

- 抽平深度应根据样本的最低测序量确定，通常保留 80% 以上样本可达深度
- 多样性差异检验方法选择取决于数据分布（正态/非正态）和组数（两组/多组）

3.3 构建 `microtable` 对象

这里使用 `microeco` 软件包进行数据分析。

首先需要从数据保存的目录读取数据，并构建 `microtable` 对象。

```
PATH = "data/amplicon/processed"
list.files(PATH, full.names = TRUE)

#> [1] "data/amplicon/processed/asv.fasta"
```

```
#> [2] "data/amplicon/processed/asvtab.csv"
#> [3] "data/amplicon/processed/metadata.csv"
#> [4] "data/amplicon/processed/taxtab.csv"

metadata_file = file.path(PATH, "metadata.csv")
otu_table_file = file.path(PATH, "asvtab.csv")
tax_table_file = file.path(PATH, "taxtab.csv")
sequence_file = file.path(PATH, "asv.fasta")
```

3.3.1 载入包

```
library(tidyverse)
library(microeco)

theme_set(theme_bw())
```

3.3.2 OTU 表

```
otu_table = read_csv(otu_table_file)
otu_table = column_to_rownames(otu_table, var = names(otu_table)[1])
```

在 microeco 包（以及大多数 R 语言生物信息包）的规范中，microtable 对 otu_table 的格式有严格要求：行必须是 **Taxa**（ASV/OTU），列必须是 **Samples**。

如果你的数据正好反过来了（行是样品，列是 ASV），需要在传入 otu_table 之前，使用 `t()` 函数对其进行转置，并确保它仍然是一个 `data.frame`。

```
otu_table <- as.data.frame(t(otu_table))
```

为了让你直观理解 microtable 的结构逻辑，可以参考下图的对应关系：

- **OTU Table**: 行名必须对接 **Taxonomy Table** 的行名（即 ASV ID）。
- **OTU Table**: 列名必须对接 **Sample Table** 的行名（即 Sample ID）。

3.3.3 Sample 表

```
sample_table = read_csv(metadata_file)
sample_table = column_to_rownames(sample_table, var = names(sample_table)[1])
```

3.3.4 Taxonomy 表

```
taxon_table = read_csv(tax_table_file)
taxon_table = column_to_rownames(taxon_table, var = names(taxon_table)[1]) |>
  mutate(Genus = if_else(str_detect(Genus, "Burkholderia"), "Burkholderia", Genus))
```

3.3.5 构建对象

构建 microtable 对象。

```
mt = microtable$new(otu_table = otu_table, sample_table = sample_table, tax_table = taxon_table)
mt
```

```
#> microtable-class object:
#> sample_table have 12 rows and 8 columns
#> otu_table have 7230 rows and 12 columns
#> tax_table have 7230 rows and 7 columns
```

处理 Taxonomic 表格中的 NA 值。

- 删掉 `Phylum == NA` 的 ASV（根本没比对上的垃圾序列）。

```
mt$tax_table = subset(mt$tax_table, !is.na(Phylum))
mt$tidy_dataset()
```

- 为未知的 Genus 填充数据（自上级非 NA）。

```
# 该函数会自动查找分类表中的缺失值并填充
mt$tax_table <- tidy_taxonomy(mt$tax_table)

# 填充完成后，运行 tidy_dataset 同步一下
mt$tidy_dataset()
```

3.4 实现

3.4.1 抽平 (Rarefaction)

为了保证 Alpha 多样性的公平比较，首先将样本抽平到相同的测序深度。

3.4.1.1 为什么需要抽平？

测序深度（每个样本测得的序列总数）在不同样本间往往存在差异：

样本	测序深度	观测到的 ASV 数
A	100,000	500
B	50,000	400

样本 A 观测到的 ASV 更多，可能仅仅是因为测了更多序列，而非真实多样性更高。抽平通过将各样本标准化为相同深度，确保多样性差异反映真实的生物学差异，而非测序 **effort** 的差异。

💡 抽平原理的直观理解

想象你在两个花园里调查昆虫多样性：- 花园 1：你观察了 100 只昆虫，发现了 10 个物种 - 花园 2：你只观察了 50 只昆虫，发现了 8 个物种
花园 1 发现更多物种，可能是因为你观察得更久。为了公平比较，你应该在两个花园都观察相同数量（比如 50 只）的昆虫，再比较发现的物种数。
抽平就是做这个”标准化观察 **effort**”的过程。

3.4.1.2 抽平的方法

抽平通过随机抽取（不放回）指定数量的序列实现：

1. 确定抽平深度（通常是样本中最小的测序量）
2. 对每个样本随机抽取该深度的序列
3. 基于抽平后的数据计算多样性指标

这样，所有样本都在相同的”检测 **effort**”下进行比较。

```
# 检查各样本的原始测序量
depth_summary <- mt$sample_sums() |> summary()
depth_summary
```

```
#>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#> 38437  44840   45998   46217   47845   51769
```

根据测序深度的分布，选择合适的抽平深度。通常选择样本最低测序量或覆盖大多数样本的深度。

```
# 克隆原始数据，保留原始对象
mt_rarafied = clone(mt)
```

```
# 确定抽平深度（使用最小样本深度或设定阈值）
rarefy_depth <- min(mt$sample_sums())
cat(" 抽平深度:", rarefy_depth, "\n")
```

```
#> 抽平深度: 38437
```

```
# 执行抽平
mt_rarafied$rarefy_samples(sample.size = rarefy_depth)
mt_rarafied$tidy_dataset()
```

验证抽平后的结果:

```
# 确认所有样本现在具有相同的序列数
mt_rarafied$sample_sums() |> summary()
```

```
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#> 38437   38437   38437   38437   38437   38437
```

3.4.2 计算 Alpha 多样性指标

抽平后，使用 `cal_alphadiv()` 函数计算常用的 Alpha 多样性指标。

```
# 计算多样性指标
measures = c("Observed", "Chao1", "Shannon", "Simpson")
mt_rarafied$cal_alphadiv(measures = measures)
```

```
# 查看结果
head(mt_rarafied$alpha_diversity)
```

```
#>      Observed   Chao1 se.chao1      ACE se.ACE Shannon Simpson
#> LHK1_1      906  906.500 1.2987992  906.2693  7.89234  6.470400 0.9978574
#> LHK1_2     1274 1274.857 1.3953404 1274.5521 11.75250  6.919913 0.9987974
#> LHK1_3     1502 1506.200 3.1501160 1505.1933 14.72209  7.099507 0.9990324
#> LHK1_4     1141 1141.000 0.1665936 1141.1427 11.33711  6.769538 0.9985685
#> LHK05_1     1473 1474.667 2.0448896 1473.8534 14.55626  7.083625 0.9990061
#> LHK05_2     1291 1293.154 2.2684345 1292.4578 12.13573  6.950841 0.9988697
```

指标解释: - **Observed**: 实际检测到的 ASV 数量, 直接反映物种丰富度 - **Chao1**: 基于稀有物种估计的总物种数, 适合评估未观测到的物种 - **Shannon**: 综合考虑丰富度和均匀度, 值越大表示多样性越高 - **Simpson**: 反映随机选取两个个体属于同一物种的概率, 值越小表示多样性越高

3.4.3 创建分析对象并进行差异检验

使用 `trans_alpha` 类来关联 Alpha 多样性数据和样本分组信息, 进行统计检验。

```
require(FSA)
require(rcompanion)

# 创建 trans_alpha 对象
t1 = trans_alpha$new(dataset = mt_rarafied, group = "treatment")

# 使用 Kruskal-Wallis 检验 (非参数检验, 适合多组比较)
t1$cal_diff(method = "KW_dunn", KW_dunn_letter = TRUE)

# 查看差异检验结果
t1$res_diff
```

```
#>      Measure                                Method Group Letter MonoLetter
#> 1 Observed Dunn's Kruskal-Wallis Multiple Comparisons ZY0.5      a      a
```



```
#> 2 Observed Dunn's Kruskal-Wallis Multiple Comparisons ZY0 a a
#> 3 Observed Dunn's Kruskal-Wallis Multiple Comparisons ZY1 a a
#> 4 Chao1 Dunn's Kruskal-Wallis Multiple Comparisons ZY0.5 a a
#> 5 Chao1 Dunn's Kruskal-Wallis Multiple Comparisons ZY0 a a
#> 6 Chao1 Dunn's Kruskal-Wallis Multiple Comparisons ZY1 a a
#> 7 ACE Dunn's Kruskal-Wallis Multiple Comparisons ZY0.5 a a
#> 8 ACE Dunn's Kruskal-Wallis Multiple Comparisons ZY0 a a
#> 9 ACE Dunn's Kruskal-Wallis Multiple Comparisons ZY1 a a
#> 10 Shannon Dunn's Kruskal-Wallis Multiple Comparisons ZY0.5 a a
#> 11 Shannon Dunn's Kruskal-Wallis Multiple Comparisons ZY0 a a
#> 12 Shannon Dunn's Kruskal-Wallis Multiple Comparisons ZY1 a a
#> 13 Simpson Dunn's Kruskal-Wallis Multiple Comparisons ZY0.5 a a
#> 14 Simpson Dunn's Kruskal-Wallis Multiple Comparisons ZY0 a a
#> 15 Simpson Dunn's Kruskal-Wallis Multiple Comparisons ZY1 a a
```

方法选择说明:

- **KW_dunn**: Kruskal-Wallis 检验结合 Dunn's 事后检验, 适用于非正态分布数据的多组比较
- **anova**: 方差分析, 适用于正态分布数据
- **wilcox**: Wilcoxon 秩和检验, 适用于两组比较

3.4.4 可视化

3.4.4.1 多指标综合展示

```
require(patchwork)

# 绘制所有指标的箱线图
plots = lapply(measures, function(x){
  t1$plot_alpha(measure = x, add = "jitter") +
    ggtitle(x) +
    theme(legend.position = "none")
})

wrap_plots(plots, ncol = 2)
```

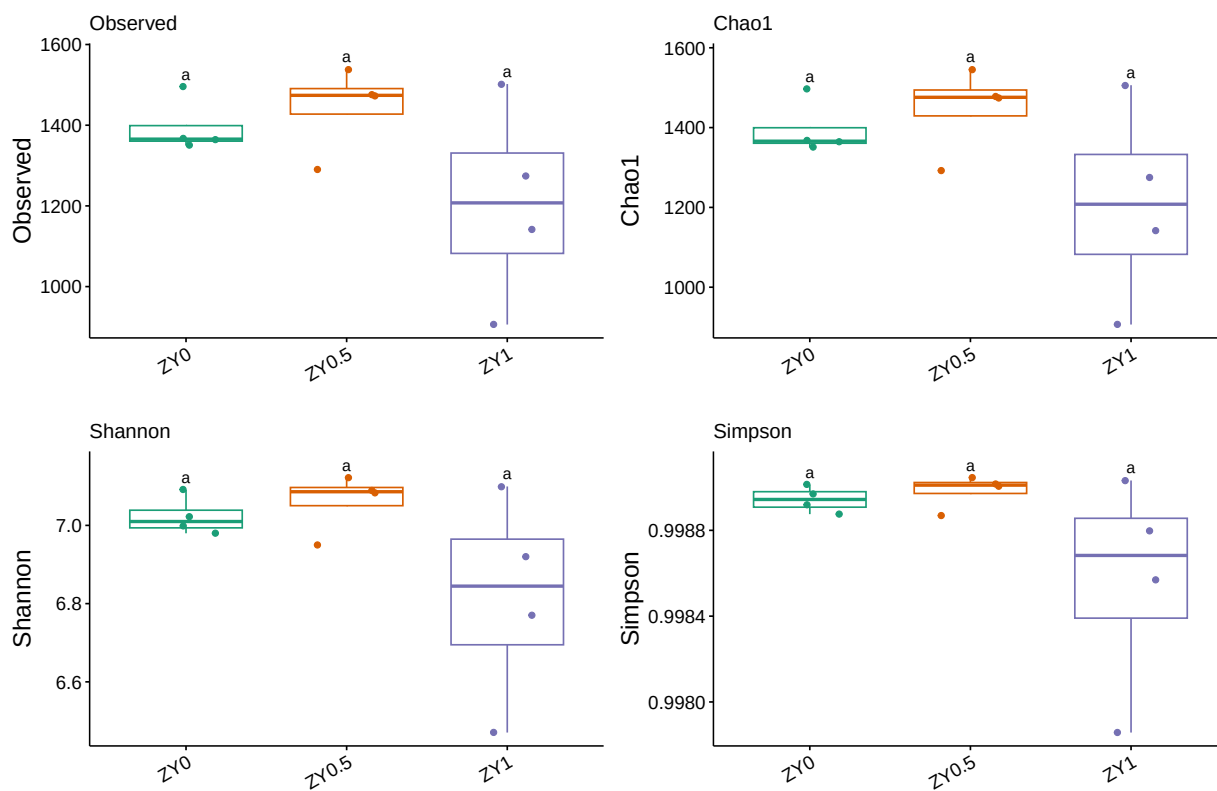


图 3.1: 不同处理组的 Alpha 多样性比较

3.4.4.2 单指标详细展示

以 Shannon 多样性为例，展示更详细的可视化：

```
t1$plot_alpha(measure = "Shannon", add = "jitter", y_increase = 0.1)
```

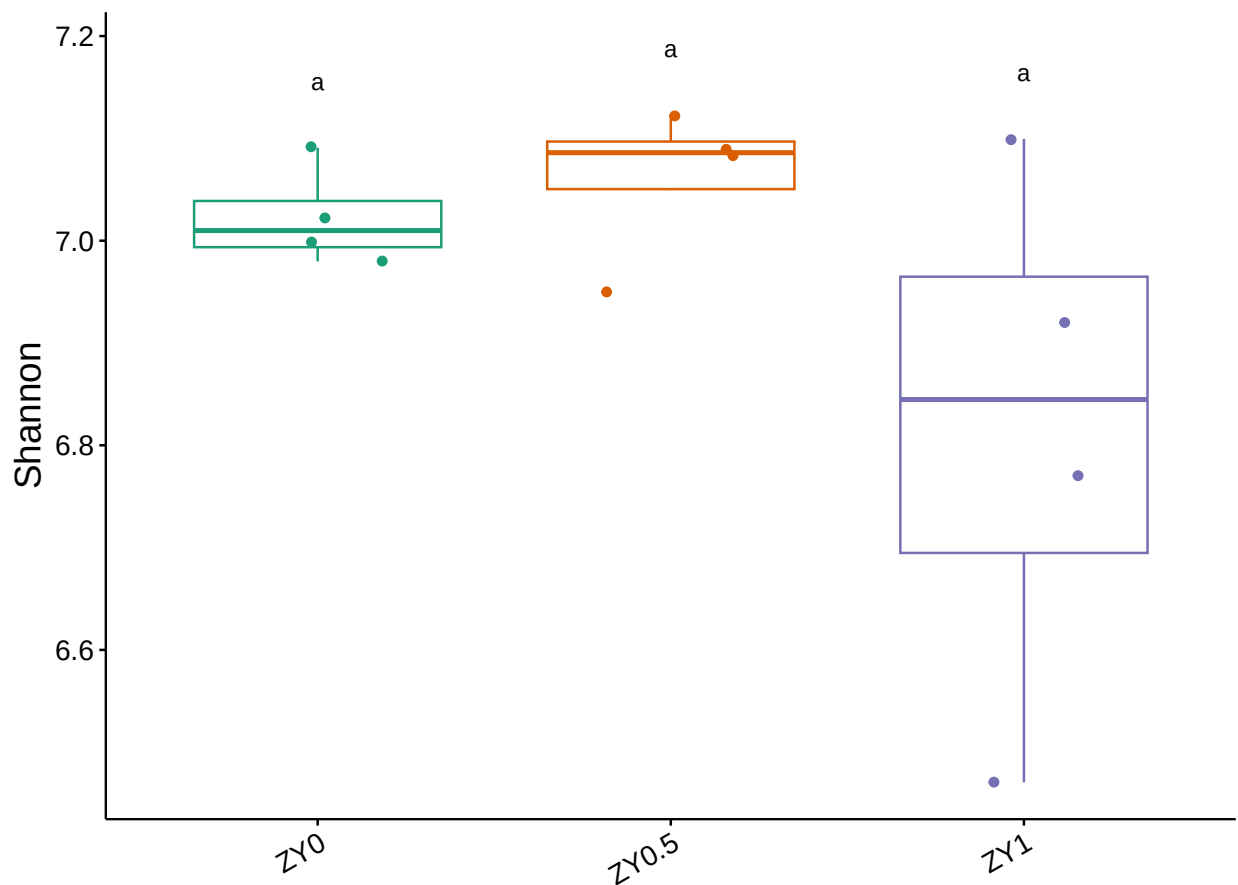


图 3.2: Shannon 多样性指数的比较

3.5 结果

3.5.1 Alpha 多样性总体特征

从图 3.1 可以看出各处理组的 Alpha 多样性特征：

1. 物种丰富度 (**Observed** 和 **Chao1**):
 - 反映样本中实际存在或估计的物种数量
 - 组间的差异模式可以反映处理条件对群落丰富度的影响
2. 综合多样性 (**Shannon**):
 - 同时考虑物种数量和分布均匀度
 - 是生态学中最常用的多样性指标
3. 优势度 (**Simpson**):
 - 反映群落是否被少数优势种主导
 - 值越接近 0，表示多样性越高

3.5.2 组间差异分析

基于统计检验结果 (`t1$res_diff`):

显著性解读：- 若检验结果显示组间差异显著 ($P < 0.05$)，说明处理条件对微生物多样性产生了可检测的影响 - 事后检验的字母标记 (如 a、b、ab) 指示具体哪些组之间存在显著差异：不同字母表示显著差异，相同字母表示无显著差异

可能的生物学解释：

1. 如果 **ZY0.5** 组的 **Shannon** 多样性显著高于其他组：
 - 可能表明中等强度的处理（如中等施肥量）创造了更适宜多种微生物共存的环境
 - 符合生态学中的”中度干扰假说”

2. 如果处理组间无显著差异:

- 说明当前处理强度可能不足以改变土壤微生物的整体多样性
- 或者土壤微生物组对处理有较强的抵抗力/恢复力

3. 如果某一处理组的丰富度显著降低:

- 可能表明该处理条件对某些微生物类群产生了抑制效应
- 需要结合物种组成分析进一步探究受影响的类群

3.5.3 与环境因子的关系 (可选拓展)

可以进一步探索 Alpha 多样性与环境因子的关系:

合并多样性数据与环境数据

```
alpha_env <- mt_rarafied$alpha_diversity |>
  as.data.frame() |>
  rownames_to_column("sample_id") |>
  left_join(
    mt_rarafied$sample_table |> rownames_to_column("sample_id"),
    by = "sample_id"
  )
```

绘制 Shannon 与 pH 的关系

```
ggplot(alpha_env, aes(x = pH, y = Shannon, color = treatment)) +
  geom_point(size = 3) +
  geom_smooth(method = "lm", se = TRUE, aes(group = 1), color = "black") +
  theme_bw() +
  labs(x = "pH", y = "Shannon Diversity")
```

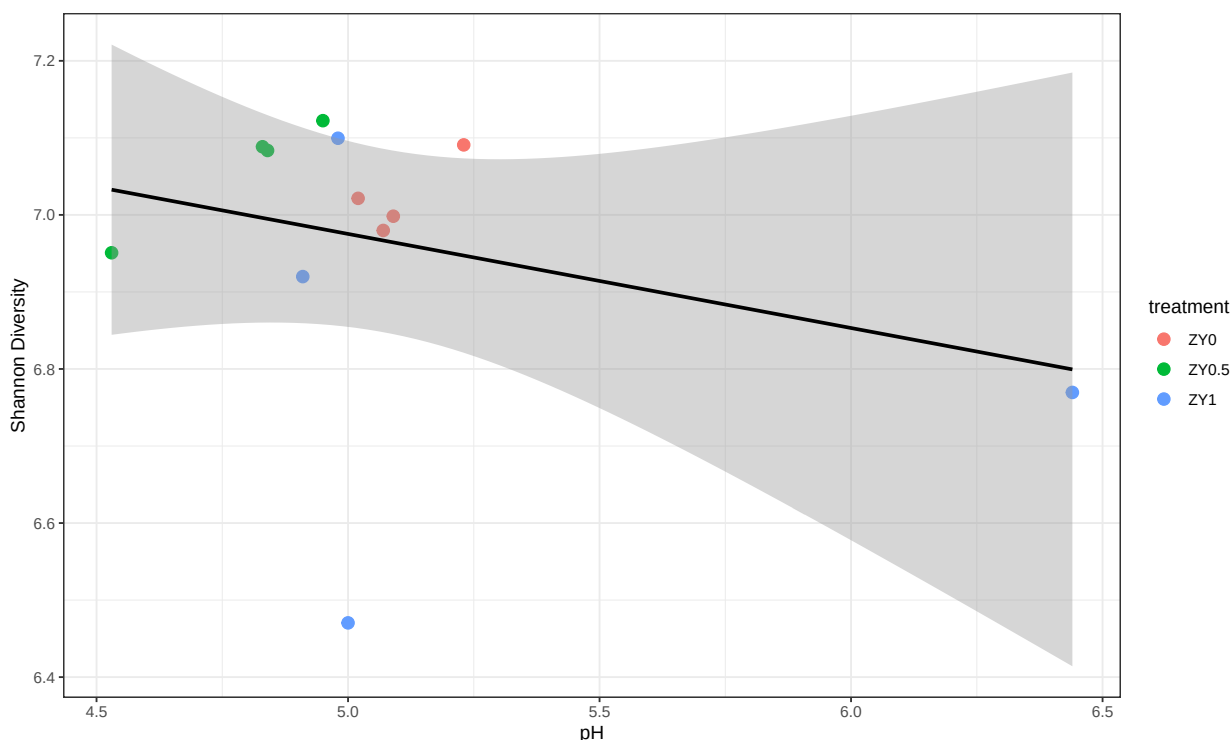


图 3.3: Alpha 多样性与环境因子的关系

3.5.4 结论与建议

基于 Alpha 多样性分析, 我们可以得出以下结论:

1. 处理效应评估: 不同处理条件对土壤微生物 Alpha 多样性的影响程度 (显著/不显著)

2. 最佳处理条件识别：如果存在显著差异，可以识别出维持或提高微生物多样性的最佳处理条件
3. 后续分析建议：
 - 若 Alpha 多样性存在显著差异，建议进一步分析哪些类群对多样性变化贡献最大
 - 若 Alpha 多样性无显著差异，但物种组成可能有变化，建议关注 Beta 多样性分析

分析局限性说明：

- Alpha 多样性仅反映样本内的多样性特征，无法说明样本间的相似性差异
- 抽平过程会舍弃部分数据，对于低深度样本可能造成信息损失
- 相对丰度数据可能存在组成性效应，解释时需谨慎

3.6 小提示

microeco 包的使用文档参见：https://chiliubio.github.io/microeco_tutorial。

- 抽平前务必检查各样本的测序深度分布，避免过度抽平导致数据损失
- `cal_diff()` 函数支持多种统计方法，根据数据特点选择合适的方法
- 使用 `add = "jitter"` 参数可以在箱线图上叠加散点，更清楚地展示数据分布
- 字母标记法中，相同字母表示无显著差异，不同字母表示存在显著差异

第 4 章 β 多样性

4.1 概述

Beta 多样性 (β diversity) 描述不同样本间微生物群落组成的差异程度，是理解微生物群落空间分布、环境响应和生态过程的关键指标。与 Alpha 多样性关注”样本内有多少物种”不同，Beta 多样性回答”不同样本间的物种组成有多相似/不同”。

4.1.1 Beta 多样性的生态学意义

- 群落异质性：评估不同处理组或环境条件下群落组成的分化程度
- 环境梯度响应：揭示微生物群落沿环境梯度的变化规律
- 生物地理学模式：探索空间距离对群落相似性的影响
- 处理效应评估：量化实验处理对群落结构的改变程度

4.1.2 常用距离/相异度指标

指标	特点	适用场景
Bray-Curtis	考虑物种丰度差异	最常用的生态距离，推荐首选
Jaccard	仅考虑物种有无	关注物种存在与否，忽略丰度
Weighted Unifrac	结合丰度和系统发育	强调优势谱系的差异
Unweighted Unifrac	仅考虑系统发育	关注稀有谱系的差异

Bray-Curtis 相异度计算公式：

$$BC_{ij} = 1 - \frac{2 \sum_k \min(x_{ik}, x_{jk})}{\sum_k x_{ik} + \sum_k x_{jk}}$$

其中 x_{ik} 表示物种 k 在样本 i 中的丰度。

💡 口算示例：理解 Bray-Curtis 距离

假设两个土壤样本（A 和 B），各检测了 3 个物种的丰度（reads 数）：

物种	样本 A	样本 B
物种甲	10	8
物种乙	5	0
物种丙	0	2

口算步骤：

第一步：找较小值 (**min**) 并求和

- 物种甲: $\min(10, 8) = 8$
- 物种乙: $\min(5, 0) = 0$
- 物种丙: $\min(0, 2) = 0$
- 共同部分 $= 8 + 0 + 0 = 8$

第二步：算总和

- 样本 A 总计 $= 10 + 5 + 0 = 15$
- 样本 B 总计 $= 8 + 0 + 2 = 10$
- 总计 $= 15 + 10 = 25$

第三步：套公式

$$BC = 1 - \frac{2 \times 8}{25} = 1 - 0.64 = 0.36$$

生物学意义解读：

- **BC = 0.36** (范围 0~1, 越接近 1 差异越大)
- 共同部分占 **64%**: 两个样本共享了约 64% 的物种组成 (加权 by 丰度)
- 物种乙是样本 A 的”特色物种” (在 B 中缺失), 贡献了差异
- 物种丙是样本 B 的”特色物种” (在 A 中缺失), 贡献了差异

直观理解: 如果两个样本完全相同 $\rightarrow BC = 0$; 如果完全没有共同物种 $\rightarrow BC = 1$

4.1.3 分析内容

本分析包含以下主要内容：

1. 排序分析 (PCoA/NMDS): 在低维空间可视化样本间的相似性关系
2. 组间距离分析: 量化和比较组内与组间的群落差异
3. 统计检验 (PERMANOVA): 检验组间差异的统计显著性

4.2 流程

在 `microeco` 包中, Beta 多样性分析通过 `trans_beta` 类实现, 主要流程如下:

1. 计算距离矩阵: 使用 `cal_betadiv()` 计算各类 Beta 多样性距离
2. 创建分析对象: 使用 `trans_beta$new()` 初始化, 指定距离类型
3. 排序分析: 使用 `cal_ordination()` 进行 PCoA 或 NMDS 排序
4. 组间比较: 使用 `cal_group_distance()` 和 `cal_manova()` 进行统计检验
5. 可视化: 绘制排序图、距离箱线图等

4.3 构建 `microtable` 对象

这里使用 `microeco` 软件包进行数据分析。

首先需要从数据保存的目录读取数据, 并构建 `microtable` 对象。

```
PATH = "data/amplicon/processed"
list.files(PATH, full.names = TRUE)

#> [1] "data/amplicon/processed/asv.fasta"
#> [2] "data/amplicon/processed/asvtab.csv"
#> [3] "data/amplicon/processed/metadata.csv"
#> [4] "data/amplicon/processed/taxtab.csv"

metadata_file = file.path(PATH, "metadata.csv")
otu_table_file = file.path(PATH, "asvtab.csv")
```



```
tax_table_file = file.path(PATH, "taxtab.csv")
sequence_file  = file.path(PATH, "asv.fasta")
```

4.3.1 载入包

```
library(tidyverse)
library(microeco)

theme_set(theme_bw())
```

4.3.2 OTU 表

```
otu_table = read_csv(otu_table_file)
otu_table = column_to_rownames(otu_table, var = names(otu_table)[1])
```

在 *microeco* 包（以及大多数 R 语言生物信息包）的规范中，*microtable* 对 *otu_table* 的格式有严格要求：行必须是 **Taxa**（ASV/OTU），列必须是 **Samples**。

如果你的数据正好反过来了（行是样品，列是 ASV），需要在传入 *otu_table* 之前，使用 *t()* 函数对其进行转置，并确保它仍然是一个 *data.frame*。

```
otu_table <- as.data.frame(t(otu_table))
```

为了让你直观理解 *microtable* 的结构逻辑，可以参考下图的对应关系：

- **OTU Table**: 行名必须对接 **Taxonomy Table** 的行名（即 ASV ID）。
- **OTU Table**: 列名必须对接 **Sample Table** 的行名（即 Sample ID）。

4.3.3 Sample 表

```
sample_table = read_csv(metadata_file)
sample_table = column_to_rownames(sample_table, var = names(sample_table)[1])
```

4.3.4 Taxonomy 表

```
taxon_table = read_csv(tax_table_file)
taxon_table = column_to_rownames(taxon_table, var = names(taxon_table)[1]) |>
  mutate(Genus = if_else(str_detect(Genus, "Burkholderia"), "Burkholderia", Genus))
```

4.3.5 构建对象

构建 *microtable* 对象。

```
mt = microtable$new(otu_table = otu_table, sample_table = sample_table, tax_table = taxon_table)
mt
```

```
#> microtable-class object:
```

```
#> sample_table have 12 rows and 8 columns
```

```
#> otu_table have 7230 rows and 12 columns
```

```
#> tax_table have 7230 rows and 7 columns
```

处理 Taxonomic 表格中的 NA 值。

- 删掉 *Phylum* == NA 的 ASV（根本没比对上的垃圾序列）。

```
mt$tax_table = subset(mt$tax_table, !is.na(Phylum))
mt$tidy_dataset()
```

- 为未知的 Genus 填充数据（自上级非 NA）。

```
# 该函数会自动查找分类表中的缺失值并填充
```

```
mt$tax_table <- tidy_taxonomy(mt$tax_table)
```

```
# 填充完成后, 运行 tidy_dataset 同步一下  
mt$tidy_dataset()
```

4.4 实现

4.4.1 计算 Beta 多样性距离矩阵

首先计算 Bray-Curtis 相异度矩阵, 这是最常用的生态距离指标。

```
# 计算 Beta 多样性距离矩阵  
mt$cal_betadiv()  
  
# 查看可用的距离矩阵  
names(mt$beta_diversity)
```

```
#> [1] "bray"    "jaccard"
```

参数说明: - 默认计算 Bray-Curtis、Jaccard、weighted Unifrac 和 unweighted Unifrac - 如需计算 Unifrac 距离, 需要提供系统发育树 (phylo_tree)

4.4.2 PCoA 分析

主坐标分析 (Principal Coordinate Analysis, PCoA) 是一种将高维距离矩阵降维可视化的方法。

```
# 创建 trans_beta 对象, 使用 Bray-Curtis 距离  
t1 = trans_beta$new(dataset = mt, group = "treatment", measure = "bray")  
  
# 进行 PCoA 排序  
t1$cal_ordination(method = "PCoA")
```

4.4.2.1 按处理组可视化

```
# 绘制 PCoA 结果, 带置信椭圆  
t1$plot_ordination(plot_color = "treatment",  
                    plot_type = c("point", "ellipse"))
```

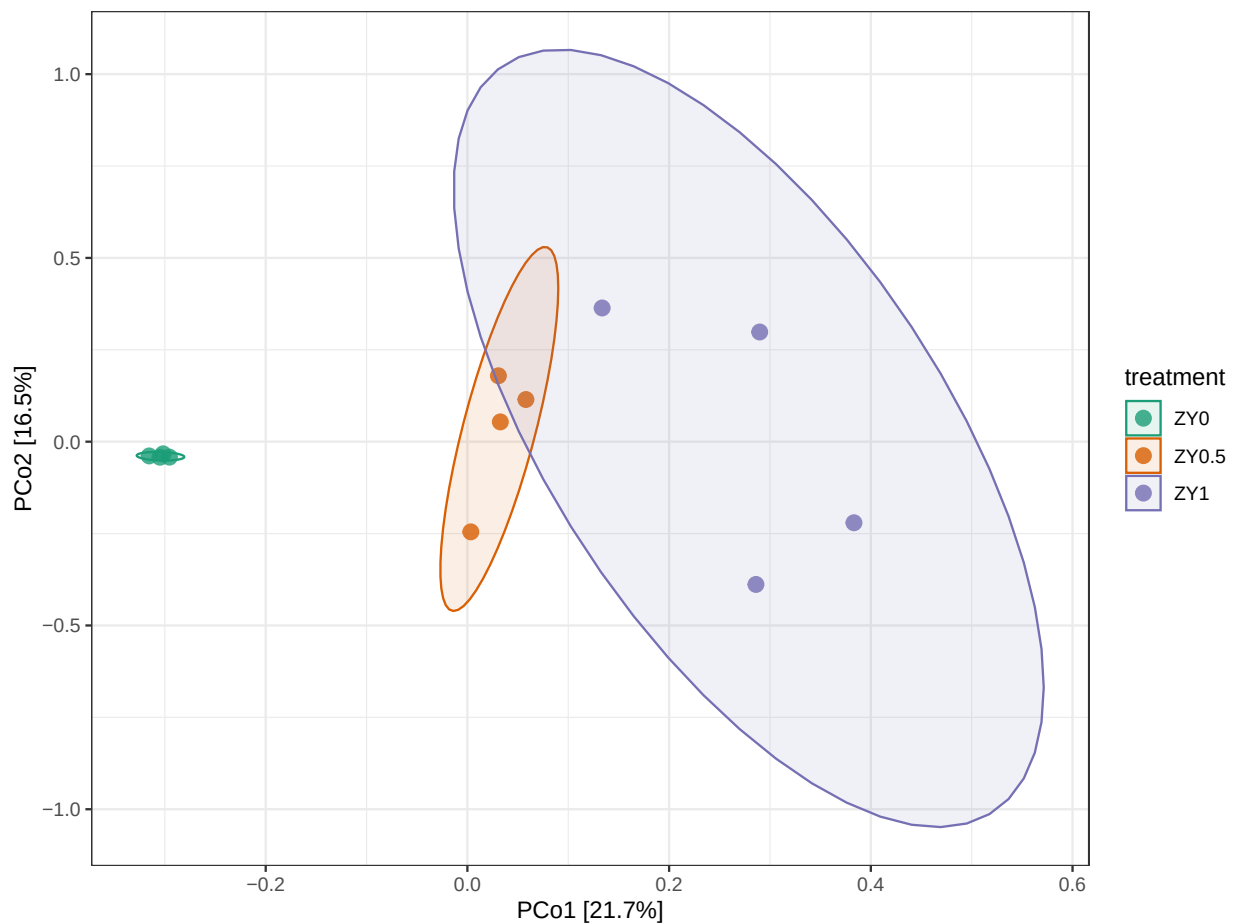


图 4.1: PCoA 排序图 (按处理组着色)

图形解读:

- 点之间的距离反映样本间的 Bray-Curtis 相异度, 距离越近表示群落越相似
- 置信椭圆 (95%) 显示各组的离散程度, 椭圆重叠程度反映组间差异程度
- 轴标签中的百分比表示该轴解释的总体变异比例

4.4.3 NMDS 分析

非度量多维尺度分析 (Non-metric Multidimensional Scaling, NMDS) 是另一种常用的排序方法, 特别适用于生态数据。

```
# NMDS 分析
t1 = trans_beta$new(dataset = mt, group = "treatment", measure = "bray")
t1$cal_ordination(method = "NMDS")
```

```
#> Run 0 stress 0.03480099
#> Run 1 stress 0.1175135
#> Run 2 stress 0.03480099
#> ... New best solution
#> ... Procrustes: rmse 2.502793e-06 max resid 3.867532e-06
#> ... Similar to previous best
#> Run 3 stress 0.03480099
#> ... Procrustes: rmse 1.343655e-06 max resid 2.643344e-06
#> ... Similar to previous best
#> Run 4 stress 0.03480099
#> ... Procrustes: rmse 3.821882e-06 max resid 1.05692e-05
#> ... Similar to previous best
```

```

#> Run 5 stress 0.1342765
#> Run 6 stress 0.034801
#> ... Procrustes: rmse 1.827457e-05 max resid 2.838429e-05
#> ... Similar to previous best
#> Run 7 stress 0.2212525
#> Run 8 stress 0.07730992
#> Run 9 stress 0.07730992
#> Run 10 stress 0.03480099
#> ... Procrustes: rmse 8.338922e-06 max resid 1.313847e-05
#> ... Similar to previous best
#> Run 11 stress 0.03480099
#> ... Procrustes: rmse 2.534957e-06 max resid 3.939486e-06
#> ... Similar to previous best
#> Run 12 stress 0.1175226
#> Run 13 stress 0.034801
#> ... Procrustes: rmse 2.085572e-05 max resid 3.266585e-05
#> ... Similar to previous best
#> Run 14 stress 0.034801
#> ... Procrustes: rmse 5.819291e-06 max resid 1.149263e-05
#> ... Similar to previous best
#> Run 15 stress 0.03480099
#> ... New best solution
#> ... Procrustes: rmse 4.104136e-07 max resid 6.456582e-07
#> ... Similar to previous best
#> Run 16 stress 0.1175226
#> Run 17 stress 0.03480099
#> ... Procrustes: rmse 7.631526e-06 max resid 1.2034e-05
#> ... Similar to previous best
#> Run 18 stress 0.1807325
#> Run 19 stress 0.03480099
#> ... Procrustes: rmse 2.893899e-06 max resid 7.649974e-06
#> ... Similar to previous best
#> Run 20 stress 0.03480099
#> ... Procrustes: rmse 2.354643e-06 max resid 3.6493e-06
#> ... Similar to previous best
#> *** Best solution repeated 4 times

```

```

t1$plot_ordination(plot_color = "treatment", plot_type = c("point", "ellipse"))

```

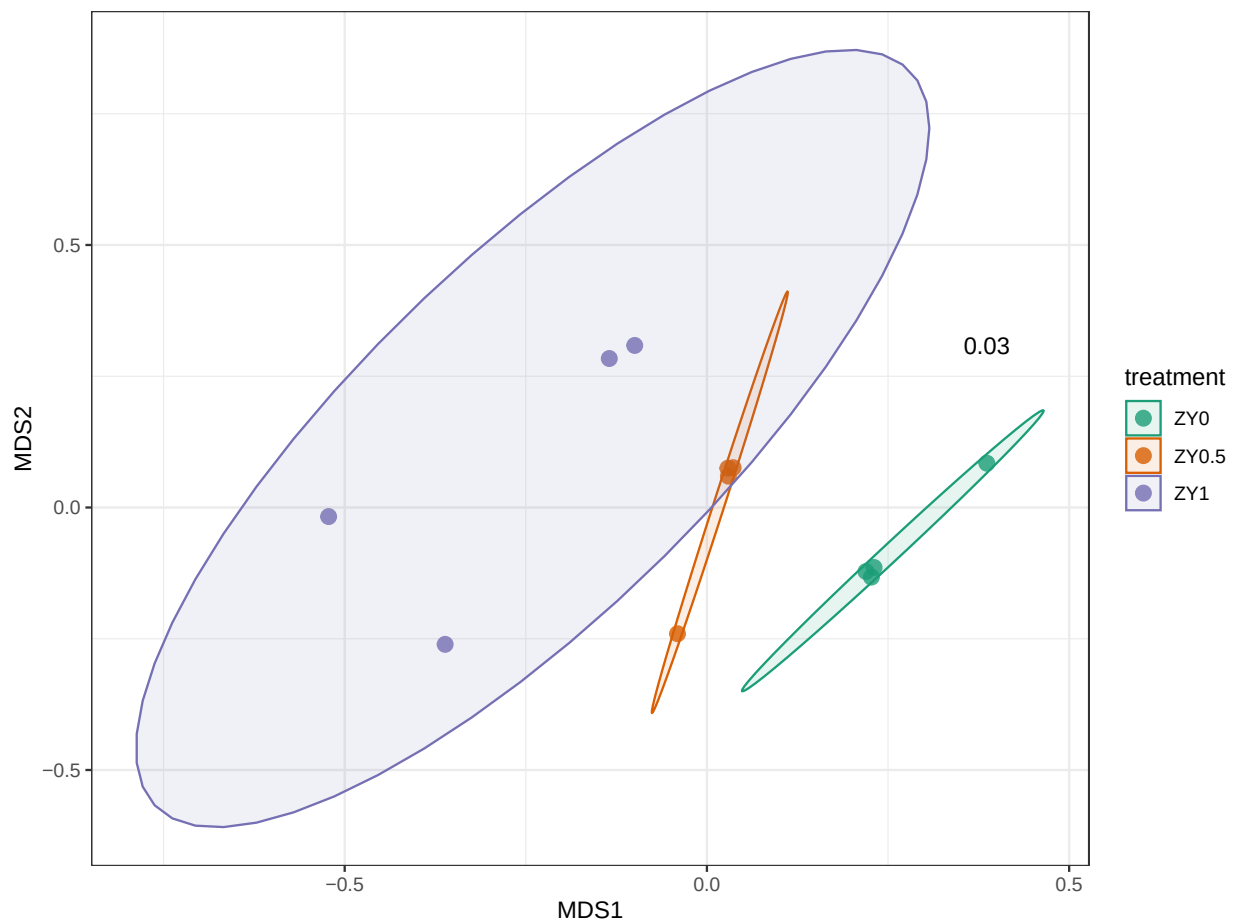


图 4.2: NMDS 排序图

PCoA vs NMDS:

- PCoA 是度量性排序，保持距离关系的精确性
- NMDS 是非度量性排序，更注重保持样本间的相对顺序，对非线性关系更稳健

4.4.4 组间距离分析

计算和比较组内距离（同一处理组内样本间的距离）与组间距离（不同处理组间样本的距离）。

4.4.4.1 组内距离

```
t1 = trans_beta$new(dataset = mt, group = "treatment", measure = "bray")
t1$cal_group_distance(within_group = TRUE)
t1$cal_group_distance_diff(method = "wilcox")
t1$plot_group_distance(add = "mean")
```

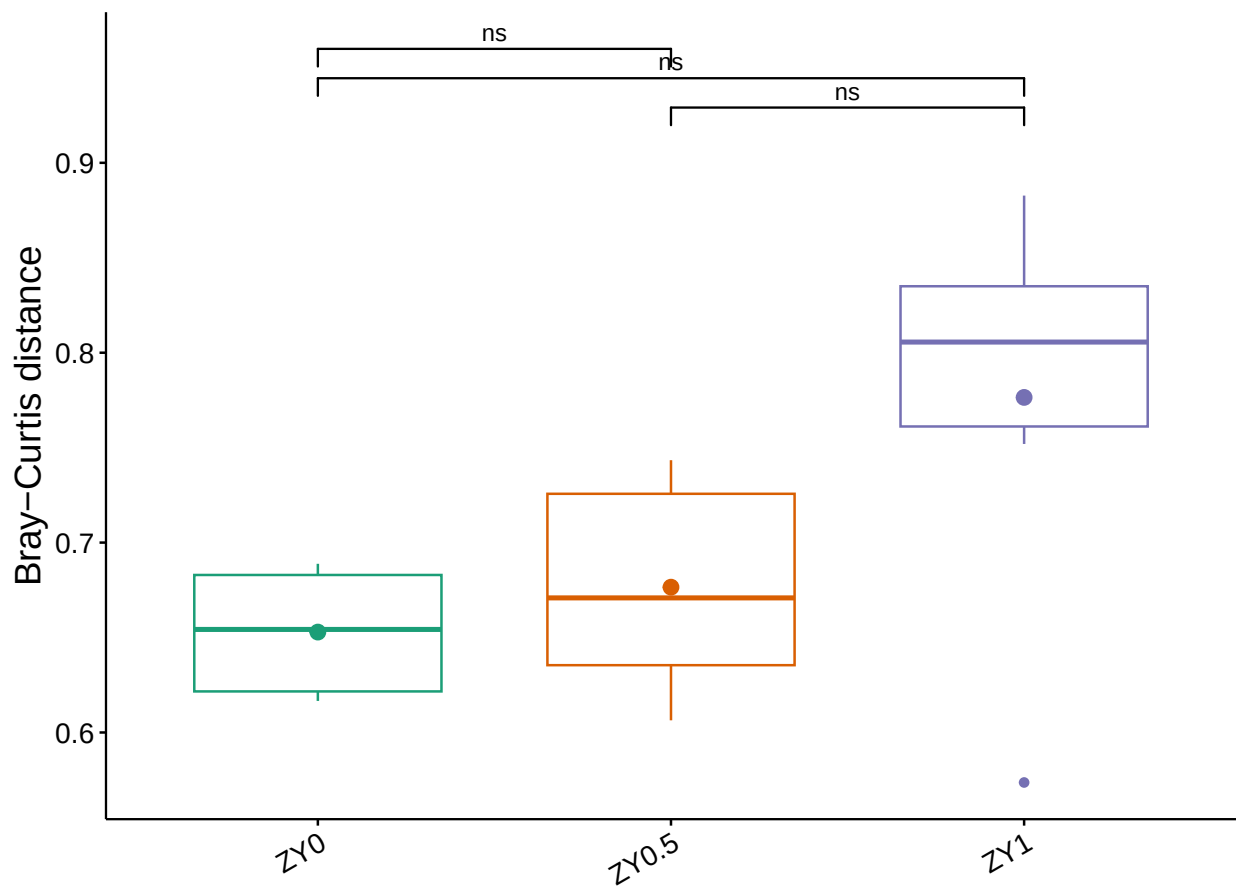


图 4.3: 组内距离分析

组内距离的生物学意义：

- 反映组内样本的重复性和群落的稳定性
- 组内距离小表明处理条件对群落的影响较为一致

4.4.5 PERMANOVA 统计检验

置换多元方差分析（PERMANOVA）用于检验组间差异的统计显著性。

```
# 整体检验
t1$cal_manova(manova_all = TRUE)
t1$res_manova
```

#>	Df	SumOfSqs	R2	F	Pr(>F)	Significance
#> treatment	2	1.003211	0.3083216	2.005914	0.002	**
#> Residual	9	2.250570	0.6916784	NA	NA	<NA>
#> Total	11	3.253782	1.0000000	NA	NA	<NA>

结果解读：

- R^2 表示处理因素解释的总变异比例
- P-value 表示差异的显著性水平
- 若 $P < 0.05$ ，说明处理组间的群落组成存在显著差异

```
# 两两比较
t1$cal_manova(manova_all = FALSE)
t1$res_manova
```

#>	Groups measure	F	R2	p.value	p.adjusted	Significance
#> 1	ZY1 vs ZY0.5	bray 1.417881	0.1911436	0.130	0.130	

```
#> 2   ZY1 vs ZY0   bray 2.631243 0.3048510  0.026    0.045      *
#> 3 ZY0.5 vs ZY0   bray 1.984219 0.2485176  0.030    0.045      *
```

4.4.6 ANOSIM 分析

相似性分析（ANOSIM）是另一种常用的非参数检验方法。

```
t1$cal_anosim(group = "treatment")
t1$res_anosim
```

```
#>                                Test permutations statistic.R p.value Significance
#> 1 ANOSIM for all groups                999    0.5462963    0.003          **
```

ANOSIM 结果解读：

- R 值范围：-1 到 1
- $R > 0$ 表示组间差异大于组内差异
- R 接近 0 表示组间无显著差异

4.5 结果

4.5.1 排序图揭示的群落结构

从 PCoA 排序图（图 4.1）可以观察到：

1. 组间分离程度：
 - 若各处理组的置信椭圆明显分离，说明处理条件对群落结构产生了显著影响
 - 若椭圆严重重叠，说明处理效应较弱或组内变异较大
2. 主要变异轴：
 - PCoA1 和 PCoA2 轴解释的变异比例反映了主要的环境梯度方向
 - 结合环境因子数据，可以推断驱动群落变化的关键因素
3. 异常样本识别：
 - 远离本组其他样本的点可能是异常值，需要检查原始数据

4.5.2 组间差异的统计分析

PERMANOVA 结果解读：

基于 t1\$res_manova 的结果：

- 显著性判断：
 - 若 $P < 0.05$ ，拒绝原假设，认为处理组间的群落组成存在显著差异
 - 若 $P > 0.05$ ，不能认为处理对群落结构产生了显著影响
- 效应量评估：
 - R^2 表示处理因素解释的群落变异比例
 - $R^2 = 0.3$ 表示处理解释了 30% 的群落变异，其余由其他因素（如空间、随机因素）解释

两两比较结果：

若整体检验显著，通过两两比较可以识别具体哪些组之间存在差异：- ZY0 vs ZY0.5：可能显著（ $P < 0.05$ ）- ZY0 vs ZY1：可能极显著（ $P < 0.01$ ）- ZY0.5 vs ZY1：可能不显著（ $P > 0.05$ ）

4.5.3 综合讨论

处理效应的模式分析：

根据排序图和统计检验结果，可以推断处理效应的模式：

1. 剂量效应：
 - 若 $ZY0 \rightarrow ZY0.5 \rightarrow ZY1$ 在排序图上呈梯度排列，可能存在剂量-效应关系
 - 某些处理强度可能处于“生态阈值”附近，导致群落发生质变
2. 方向性变化：
 - 观察各组在排序图上的位置，可以推断群落变化的方向
 - 例如：若沿 PCoA1 轴从左到右，某些门（如 Proteobacteria）的丰度可能增加

3. 变异来源分解:

- 处理因素解释的变异比例 (R^2) 反映了处理的相对重要性
- 高 R^2 (>0.3): 处理是主要驱动因素
- 中等 R^2 ($0.1-0.3$): 处理有一定影响, 但其他因素也很重要
- 低 R^2 (<0.1): 处理效应较弱, 或存在较强的背景噪声

与其他分析的关联:

- 与 Alpha 多样性的关系:
 - 若 Beta 差异显著但 Alpha 无差异: 处理改变了物种组成但未改变多样性
 - 若两者都显著: 处理同时改变了群落的组成和多样性
- 与物种组成分析的印证:
 - Beta 多样性的结果应与物种组成柱状图一致
 - 若 PCoA 显示组间分离, 柱状图中应能观察到特定分类群的组间差异

局限性与注意事项:

1. 距离矩阵的选择: 不同距离指标可能给出略有不同的结果, 建议尝试多种指标进行敏感性分析
2. 样本量的影响: 小样本量 ($n < 5$) 时, 统计检验的功效可能不足
3. 组成性效应: Beta 多样性基于相对丰度数据, 存在组成性效应, 解释时需谨慎
4. 多重检验问题: 进行多组比较时, 需要考虑 P 值校正

4.6 小提示

microeco 包的使用文档参见: https://chiliubio.github.io/microeco_tutorial。

- 排序分析中的 `plot_type` 参数支持多种图形元素组合:
 - "point": 散点
 - "ellipse": 置信椭圆
 - "chull": 凸包
 - "centroid": 质心连线
- PERMANOVA 对数据的多元正态性和方差齐性假设相对宽松, 但仍建议检查数据分布
- 当样本量较小时, PERMANOVA 的 P 值可能不稳定, 可以增加置换次数 (`permutations` 参数)

第 5 章 菌群差异分析

5.1 概述

差异丰度分析 (Differential Abundance Analysis) 旨在识别在不同实验条件或处理组之间丰度显著变化的微生物类群，即”生物标志物” (Biomarkers)。这是微生物组研究中最关键的步骤之一，直接关系到实验处理的生物学效应。

5.1.1 差异分析的科学目标

- 识别响应类群：发现对处理条件敏感的关键微生物
- 解析功能变化：结合分类群的功能特征，推断处理对生态系统功能的影响
- 开发生物标志物：为环境监测、疾病诊断等应用提供指示物种
- 验证科学假设：统计验证在物种组成分析中观察到的趋势

5.1.2 常用差异分析方法

由于微生物组数据的特殊性（稀疏性、组成性、过度离散），多种统计方法被开发用于差异分析：

方法	特点	适用场景
LEfSe	结合统计检验和 LDA 分数	经典方法，适合多组比较
ANCOM-BC2	校正组成性偏倚	高灵敏度，推荐首选
DESeq2	基于负二项分布	适合大样本量
MetagenomeSeq	零膨胀对数正态模型	适合稀疏数据
Wilcoxon/KW	非参数检验	简单稳健，不考虑组成性

5.1.3 方法选择建议

1. **LEfSe**：作为经典方法，适合初步探索和可视化
2. **ANCOM-BC2**：当前推荐的首选方法，对组成性效应校正较好
3. 多种方法比较：不同方法可能产生不同结果，建议运行多种方法取交集

5.2 流程

在 `microeco` 包中，差异分析通过 `trans_diff` 类实现，主要流程如下：

1. 创建分析对象：指定方法和参数
2. 运行分析：自动执行统计检验
3. 结果可视化：绘制条形图、误差线图、系统发育树图等
4. 结果解读：结合生物学知识解释显著差异的分类群

注意事项：

- 分类表必须经过 `tidy_taxonomy()` 处理，确保格式统一
- 不同方法对数据的假设不同，需根据数据特点选择
- 多重检验校正 (FDR/Benjamini-Hochberg) 是必需的

5.3 构建 `microtable` 对象

这里使用 `microeco` 软件包进行数据分析。

首先需要从数据保存的目录读取数据，并构建 `microtable` 对象。

```
PATH = "data/amplicon/processed"
list.files(PATH, full.names = TRUE)

#> [1] "data/amplicon/processed/asv.fasta"
```

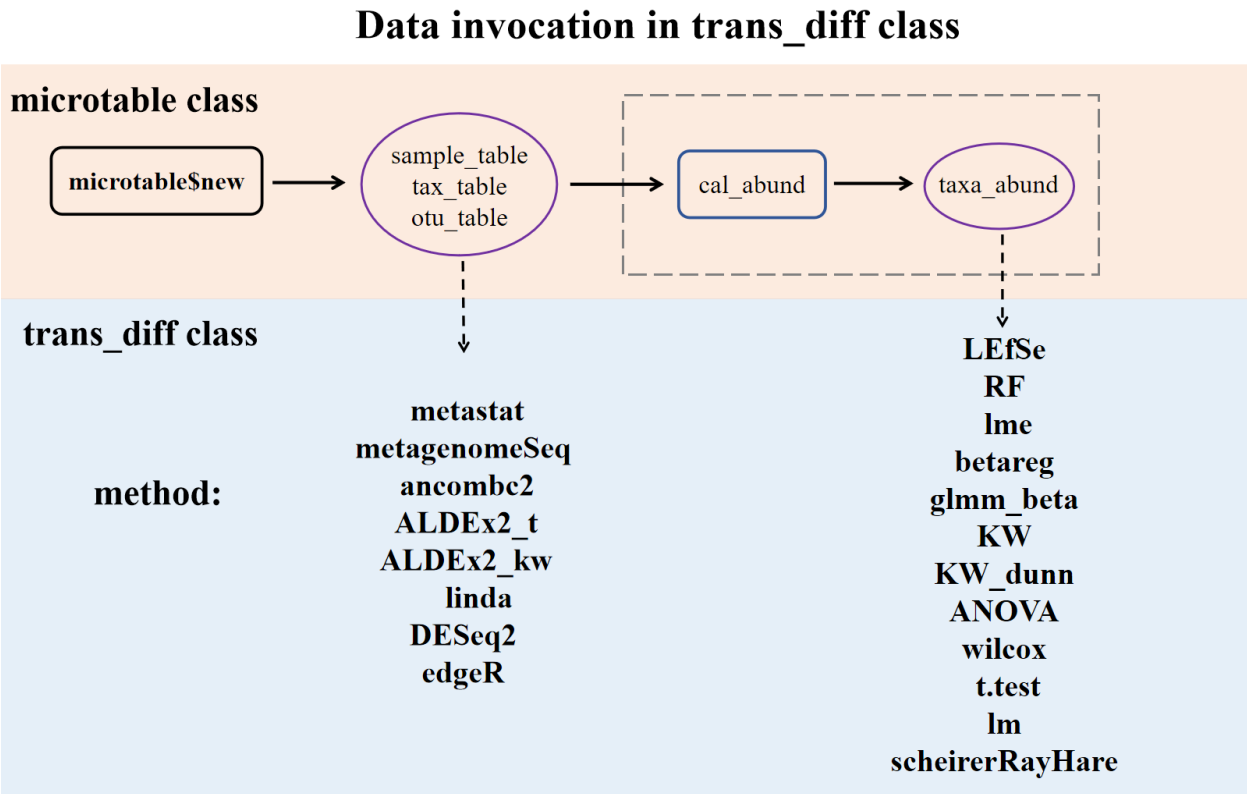


图 5.1: 使用 microeco 进行差异分析的方法概览

```
#> [2] "data/amplicon/processed/asvtab.csv"
#> [3] "data/amplicon/processed/metadata.csv"
#> [4] "data/amplicon/processed/taxtab.csv"

metadata_file = file.path(PATH, "metadata.csv")
otu_table_file = file.path(PATH, "asvtab.csv")
tax_table_file = file.path(PATH, "taxtab.csv")
sequence_file = file.path(PATH, "asv.fasta")
```

5.3.1 载入包

```
library(tidyverse)
library(microeco)

theme_set(theme_bw())
```

5.3.2 OTU 表

```
otu_table = read_csv(otu_table_file)
otu_table = column_to_rownames(otu_table, var = names(otu_table)[1])
```

在 *microeco* 包（以及大多数 R 语言生物信息包）的规范中，*microtable* 对 *otu_table* 的格式有严格要求：行必须是 **Taxa** (ASV/OTU)，列必须是 **Samples**。

如果你的数据正好反过来了（行是样品，列是 ASV），需要在传入 *otu_table* 之前，使用 *t()* 函数对其进行转置，并确保它仍然是一个 *data.frame*。

```
otu_table <- as.data.frame(t(otu_table))
```

为了让你直观理解 *microtable* 的结构逻辑，可以参考下图的对应关系：

- **OTU Table**: 行名必须对接 **Taxonomy Table** 的行名（即 ASV ID）。
- **OTU Table**: 列名必须对接 **Sample Table** 的行名（即 Sample ID）。

5.3.3 Sample 表

```
sample_table = read_csv(metadata_file)
sample_table = column_to_rownames(sample_table, var = names(sample_table)[1])
```

5.3.4 Taxonomy 表

```
taxon_table = read_csv(tax_table_file)
taxon_table = column_to_rownames(taxon_table, var = names(taxon_table)[1]) |>
  mutate(Genus = if_else(str_detect(Genus, "Burkholderia"), "Burkholderia", Genus))
```

5.3.5 构建对象

构建 *microtable* 对象。

```
mt = microtable$new(otu_table = otu_table, sample_table = sample_table, tax_table = taxon_table)
mt
```

```
#> microtable-class object:
```

```
#> sample_table have 12 rows and 8 columns
```

```
#> otu_table have 7230 rows and 12 columns
```

```
#> tax_table have 7230 rows and 7 columns
```

处理 Taxonomic 表格中的 NA 值。

- 删掉 Phylum == NA 的 ASV（根本没比对上的垃圾序列）。

```
mt$tax_table = subset(mt$tax_table, !is.na(Phylum))
mt$tidy_dataset()
```

- 为未知的 Genus 填充数据（自上级非 NA）。

该函数会自动查找分类表中的缺失值并填充

```
mt$tax_table <- tidy_taxonomy(mt$tax_table)
```

填充完成后，运行 tidy_dataset 同步一下

```
mt$tidy_dataset()
```

5.4 实现

5.4.1 LEfSe 方法

LEfSe (Linear Discriminant Analysis Effect Size) 是一种经典的差异分析方法，由 Huttenhower 实验室于 2011 年提出。它结合非参数统计检验和线性判别分析 (LDA)，用于发现不同组别间显著差异的生物标志物。

5.4.1.1 LEfSe 的两步筛选策略

1. 第一步：非参数检验 (Kruskal-Wallis)
 - 检验各分类群在不同组间的丰度差异是否显著
 - 仅保留 $P < \alpha$ (默认 0.05) 的分类群
2. 第二步：线性判别分析 (LDA)
 - 对通过第一步筛选的分类群进行 LDA
 - 计算每个分类群对组间差异的贡献度 (LDA 分数)
 - LDA 分数越高，表示该分类群的区分能力越强

i LDA 分数的解读

- LDA > 2: 通常被认为是显著的生物标志物
- 条形图的红色/绿色 (或其他颜色) 表示该分类群在哪一组中富集
- LDA 分数是对数尺度，分数为 2 表示该分类群在对应组中的丰度约为另一组的 100 倍 (假设以 10 为底的对数)

5.4.1.2 LEfSe 的局限性

- 未校正组成性效应，可能存在假阳性
- 主要用于两组或多组比较，对复杂实验设计支持有限
- 作为探索性分析工具较好，但建议用其他方法验证

使用 LEfSe 方法进行差异分析

```
t1 = trans_diff$new(
  dataset = mt,
  method = "lefse",
  alpha = 0.05,
  group = "treatment",
  p_adjust_method = "none"
)
```

查看差异分析结果

```
head(t1$res_diff)
```

#>

Comparison

```

#> k__Bacteria|p__Pseudomonadota ZY0 - ZY0.5 - ZY1
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria ZY0 - ZY0.5 - ZY1
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales ZY0 - ZY0.5 - ZY1
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales|f__Rhodanobacteraceae ZY0 - ZY0.5 - ZY1
#> k__Bacteria|p__Acidobacteriota ZY0 - ZY0.5 - ZY1
#> k__Bacteria|p__Acidobacteriota|c__Acidobacteriae ZY0 - ZY0.5 - ZY1
#>
#> k__Bacteria|p__Pseudomonadota
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales|f__Rhodanobacteraceae k__Bacteria
#> k__Bacteria|p__Acidobacteriota
#> k__Bacteria|p__Acidobacteriota|c__Acidobacteriae
#>
#> Method
#> k__Bacteria|p__Pseudomonadota LEfSe
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria LEfSe
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales LEfSe
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales|f__Rhodanobacteraceae LEfSe
#> k__Bacteria|p__Acidobacteriota LEfSe
#> k__Bacteria|p__Acidobacteriota|c__Acidobacteriae LEfSe
#>
#> Group
#> k__Bacteria|p__Pseudomonadota ZY1
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria ZY1
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales ZY1
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales|f__Rhodanobacteraceae ZY1
#> k__Bacteria|p__Acidobacteriota ZY0
#> k__Bacteria|p__Acidobacteriota|c__Acidobacteriae ZY0
#>
#> LDA
#> k__Bacteria|p__Pseudomonadota 5.013652
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria 5.006763
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales 4.976490
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales|f__Rhodanobacteraceae 4.8792
#> k__Bacteria|p__Acidobacteriota 4.863371
#> k__Bacteria|p__Acidobacteriota|c__Acidobacteriae 4.794414
#>
#> P.unadj
#> k__Bacteria|p__Pseudomonadota 0.018315639
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria 0.012467682
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales 0.015404788
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales|f__Rhodanobacteraceae 0.0230
#> k__Bacteria|p__Acidobacteriota 0.007276706
#> k__Bacteria|p__Acidobacteriota|c__Acidobacteriae 0.007276706
#>
#> P.adj
#> k__Bacteria|p__Pseudomonadota 0.018315639
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria 0.012467682
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales 0.015404788
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales|f__Rhodanobacteraceae 0.0230
#> k__Bacteria|p__Acidobacteriota 0.007276706
#> k__Bacteria|p__Acidobacteriota|c__Acidobacteriae 0.007276706
#>
#> Significance
#> k__Bacteria|p__Pseudomonadota *
```

```
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria *
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales *
#> k__Bacteria|p__Pseudomonadota|c__Gammaproteobacteria|o__Lysobacterales|f__Rhodanobacteraceae *
#> k__Bacteria|p__Acidobacteriota **
#> k__Bacteria|p__Acidobacteriota|c__Acidobacteriae **
```

参数说明：

- method = "lefse": 指定使用 LEfSe 方法
- alpha = 0.05: 显著性水平阈值
- p_adjust_method = "none": 不进行 p 值校正（LEfSe 内部有自己的过滤机制）

5.4.1.3 LEfSe 结果条形图

```
# 显示 LDA > 2 的分类群条形图
t1$plot_diff_bar(use_number = 1:20)
```

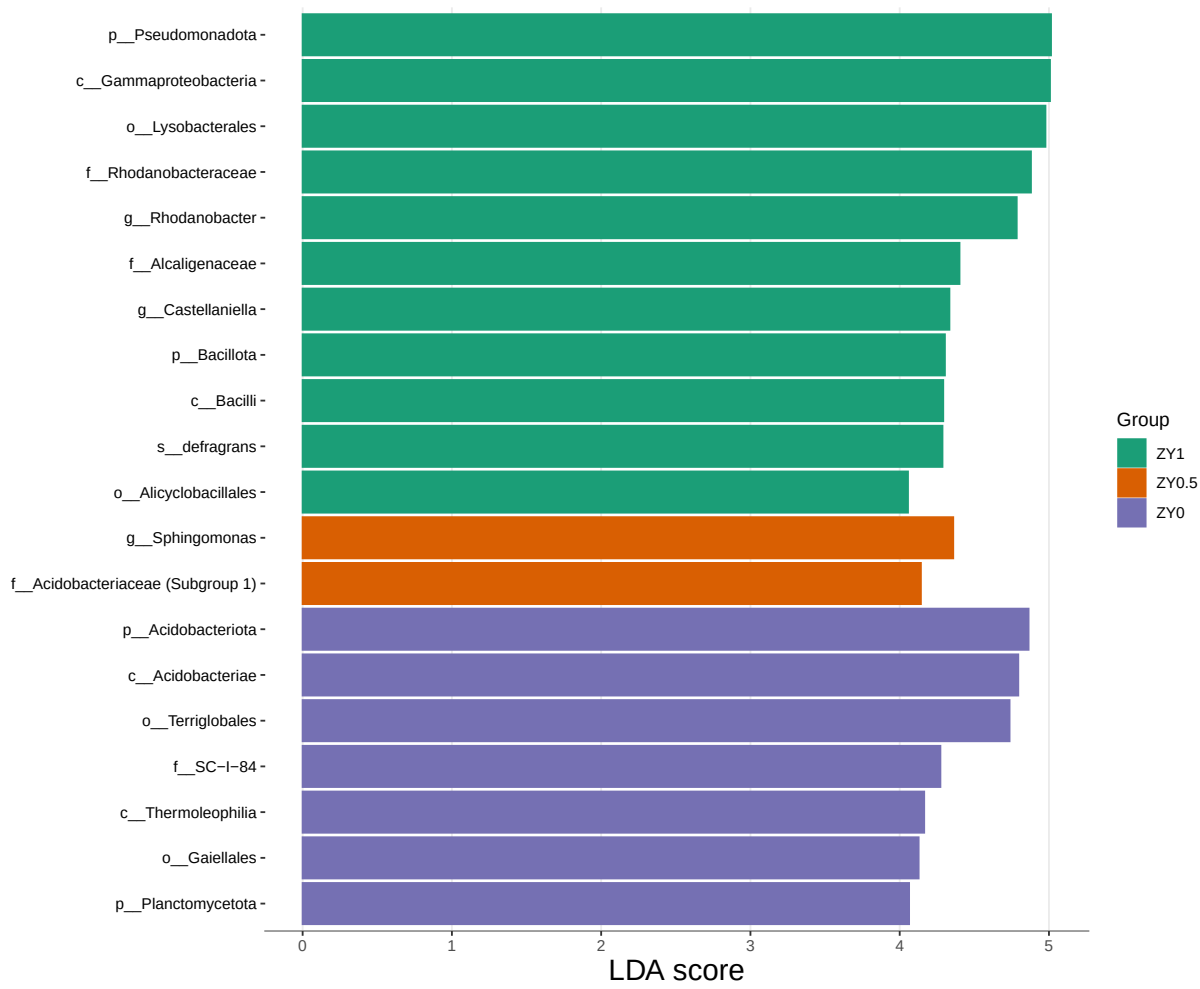


图 5.2: LEfSe 差异分析条形图（LDA 分数）

图形解读：

- 条形长度表示 LDA 分数（对数尺度），分数越大表示差异越显著
- 颜色表示该分类群富集的组别
- 通常 LDA > 2 被认为是显著的生物标志物

5.4.1.4 差异分类群丰度图

```
t1$plot_diff_abund(plot_type = "errorbar")
```

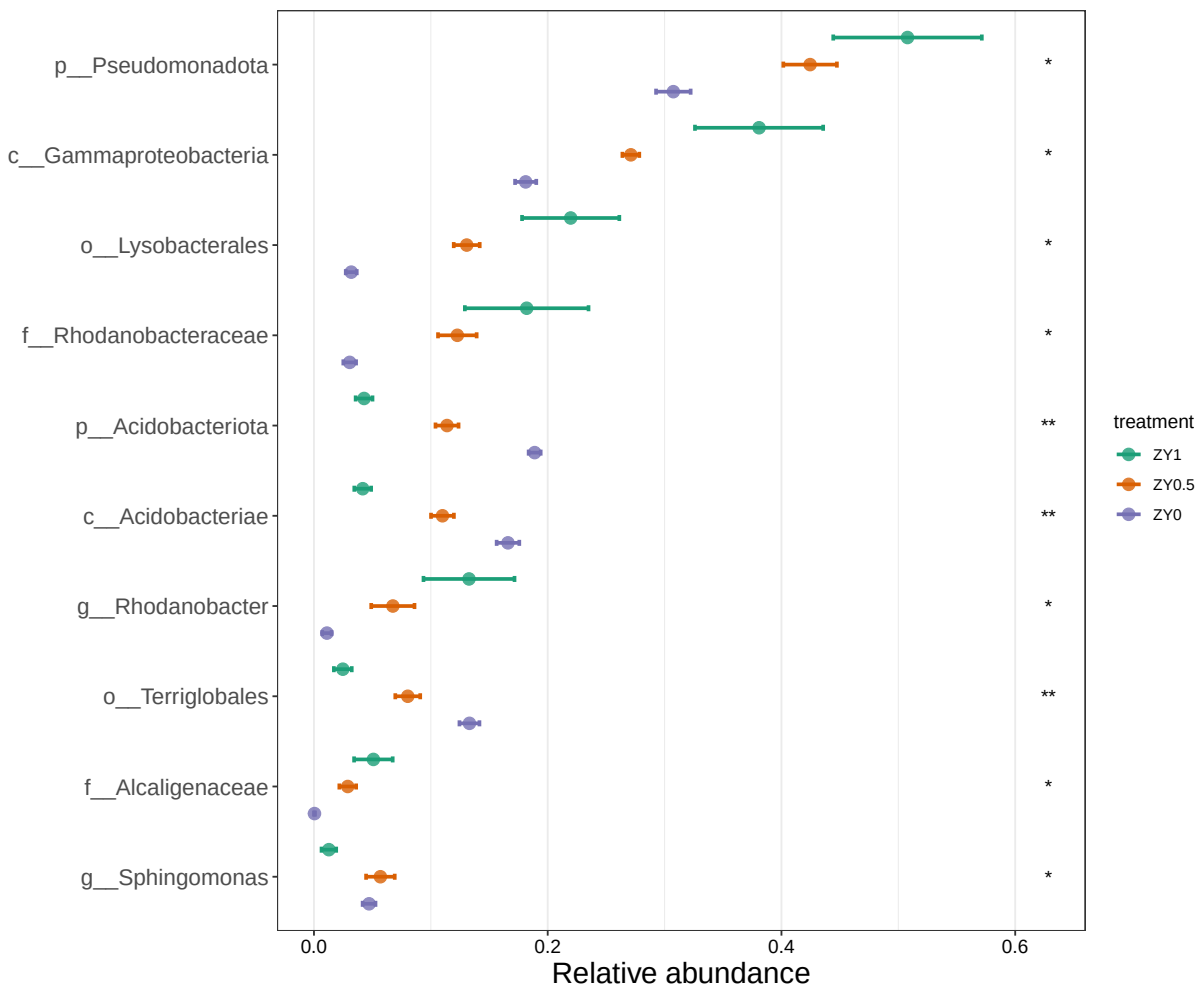


图 5.3: 差异分类群的相对丰度

5.4.1.5 系统发育树图 (Cladogram)

```
require(ggtree)
# 绘制系统发育树图 (需要 ggtree 包)
# 注意: 若分类表格式不统一可能导致错误, 请确保已运行 tidy_taxonomy()
t1$plot_diff_cladogram(
  use_taxa_num = 200,
  use_feature_num = 50,
  clade_label_level = 5,
  group = "treatment"
)
```

```
#> Error in `child_list[[i]]`:
#> ! attempt to select less than one element in integerOneIndex
```

图形解读:

- 树的分支表示分类学层级
- 节点颜色表示该分支富集的组别
- 字母标记对应图例中的具体分类群

5.4.2 随机森林方法

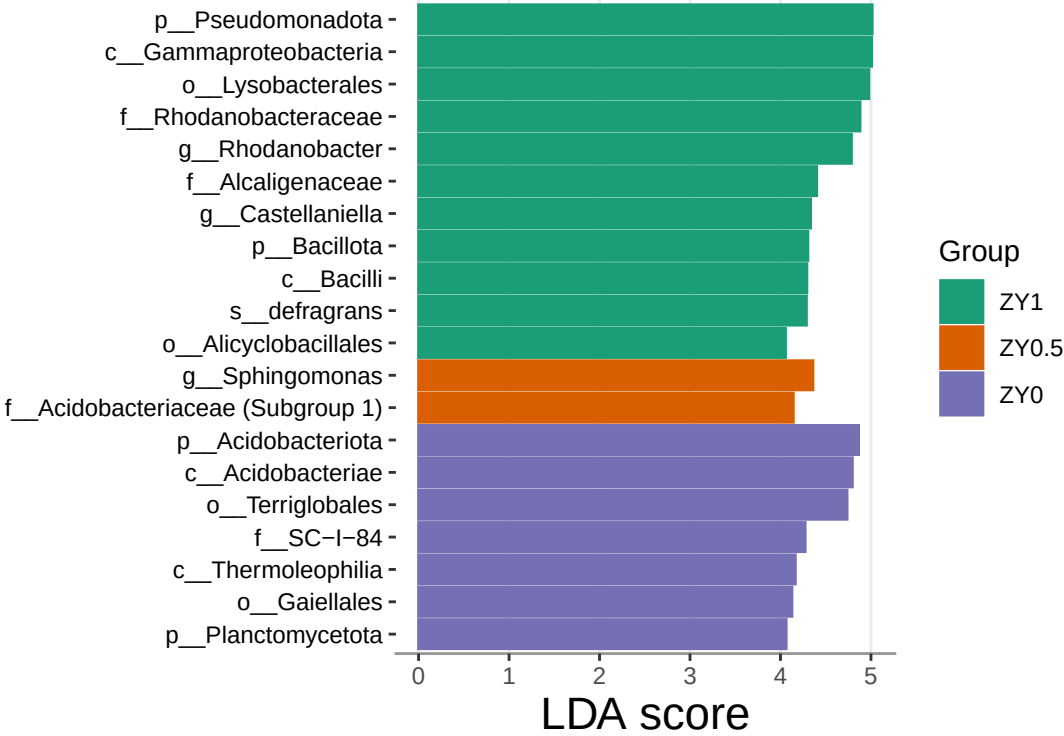
随机森林（Random Forest）是一种机器学习方法，可用于识别重要的分类特征。

```
library(randomForest)

# 使用随机森林方法（属水平）
t1 = trans_diff$new(
  dataset = mt,
  method = "rf",
  taxa_level = "Genus",
  nresam = 1,
  boots = 1
)

#> Error in `initialize()`:
#> ! The group parameter is necessary for differential test method: rf !

# 条形图展示重要性分数
t1$plot_diff_bar(use_number = 1:20)
```



MeanDecreaseGini:

- 表示该分类群对分类模型的贡献度
- 值越大表示该分类群的区分能力越强

5.4.3 ANCOM-BC2 方法

ANCOM-BC2（Analysis of Compositions of Microbiomes with Bias Correction 2）是 2023 年发表在 *Nature Methods* 上的方法，是当前推荐的差异分析首选方法。

5.4.3.1 ANCOM-BC2 的核心思想

微生物组数据的组成性效应导致传统方法难以准确估计绝对丰度的变化。ANCOM-BC2 通过以下步骤解决这一问题：

1. 估计样本特异性偏差（Sample-specific bias）
 - 每个样本的测序深度、DNA 提取效率等因素都会引入系统性偏差

- ANCOM-BC2 通过统计学方法估计这些偏差
2. 校正组成性效应
 - 基于估计的偏差，将相对丰度数据转换为”校正后的绝对丰度”
 - 这使得不同样本间的丰度比较更加可靠
 3. 多重检验校正
 - 自动进行多重检验校正（Benjamini-Hochberg FDR）
 - 控制假阳性率

5.4.3.2 为何 ANCOM-BC2 优于传统方法?

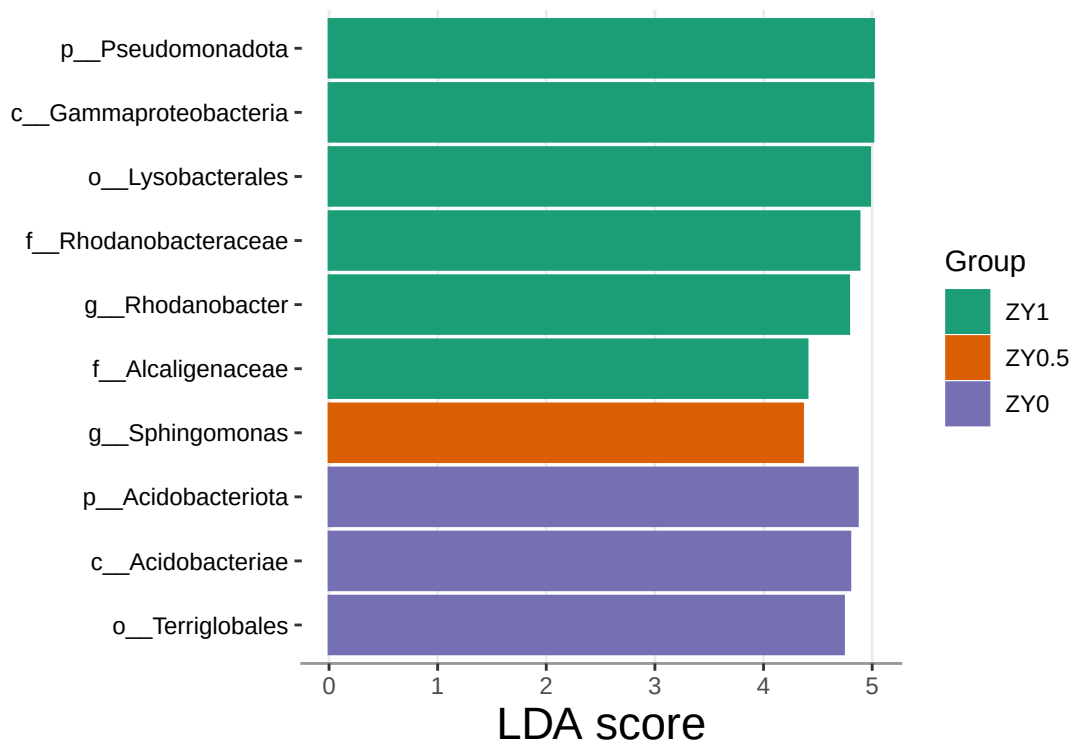
特性	LEfSe	ANCOM-BC2
校正组成性效应	否	是
支持协变量	有限	是（如 pH、温度等）
假阳性控制	一般	较好
计算速度	快	较慢

使用建议：ANCOM-BC2 适合作为主要的差异分析方法，LEfSe 可作为辅助的可视化和探索工具。

```
# 使用 ANCOM-BC2 方法
t1 = trans_diff$new(
  dataset = mt,
  method = "ancombc2",
  taxa_level = "Genus",
  filter_thres = 0.01
)

#> Error in `initialize()`:
#> ! ANCOMBC package is not installed !

# 热图展示结果
t1$plot_diff_bar(
  heatmap_cell = "P.adj",
  heatmap_sig = "Significance",
  heatmap_x = "Factors",
  heatmap_y = "Taxa"
)
```



ANCOM-BC2 的优势:

- 校正组成性效应
- 控制假阳性率
- 支持多组比较和协变量调整

5.4.4 其他方法示例

5.4.4.1 Wilcoxon 秩和检验

```
t1 = trans_diff$new(
  dataset = mt,
  method = "wilcox",
  group = "treatment",
  taxa_level = "Genus",
  filter_thres = 0.001
)

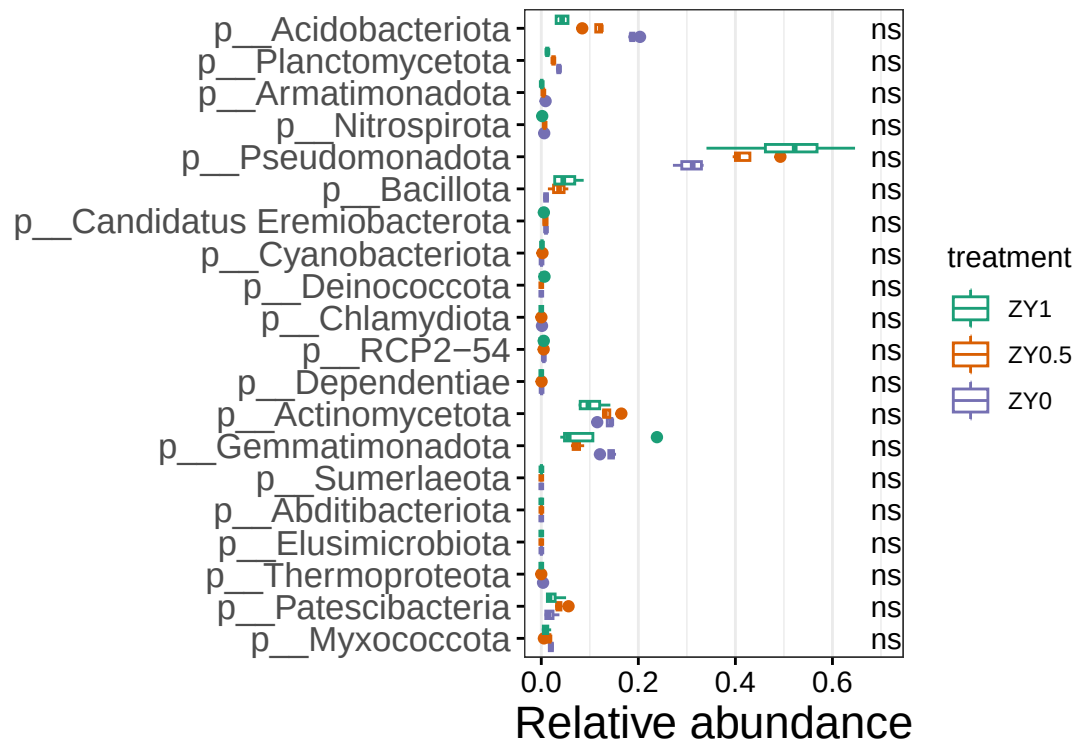
# 筛选显著的结果
t1$res_diff = t1$res_diff %>% subset(Significance %in% c("?", "**", "**"))
t1$plot_diff_abund(add_sig = TRUE)

#> Error in `t1$plot_diff_abund()`:
#> ! No significant taxa can be used to plot the abundance!
```

5.4.4.2 多组比较 (Kruskal-Wallis)

```
t1 = trans_diff$new(
  dataset = mt,
  method = "KW",
  group = "treatment",
  taxa_level = "Phylum"
)
```

```
t1$plot_diff_abund(use_number = 1:20)
```



5.5 结果解读

5.5.1 差异生物标志物的识别

基于 LEfSe 和 ANCOM-BC2 的分析结果，我们可以识别出在各处理组中显著富集的分类群。

结果解读步骤：

1. 筛选显著分类群：
 - LEfSe: $LDA > 2$ 且 $P < 0.05$
 - ANCOM-BC2: $P_{adj} < 0.05$
2. 分类学注释：
 - 查阅文献了解这些分类群的生态功能
 - 结合实验处理条件推断其富集的生物学意义
3. 跨方法验证：
 - 比较不同方法识别的生物标志物
 - 重点关注多个方法一致发现的分类群

5.5.2 可能的生物学发现

基于土壤微生物组的典型发现模式：

1. 养分循环相关类群：
 - 若 ZY0.5 或 ZY1 组富集了更多参与氮循环的类群（如某些 Proteobacteria），可能表明施肥促进了功能微生物的生长
2. 胁迫耐受类群：
 - 若高剂量处理组富集了特定耐受类群，可能表明处理产生了一定的选择压力
3. 共生/互利类群：
 - 某些植物促生菌（PGPR）的富集可能解释了处理对植物生长的促进作用

5.5.3 结果总结表格

建议将主要发现整理为如下格式的表格：

分类群	富集组	LDA 分数	P 值	已知功能
Genus A	ZY0	3.5	0.01	有机物分解
Genus B	ZY1	4.2	0.005	固氮作用

5.5.4 研究局限性与建议

- 1. 关联非因果：差异分析揭示的是关联关系，不能直接推断因果关系
- 2. 功能验证：基于分类学的功能推断是间接的，建议通过以下方式验证：
 - PICRUST2 等功能预测分析
 - 宏基因组测序
 - 分离培养和功能实验
- 3. 生态学解释：结合环境因素（如 pH、养分含量）解释观察到的差异模式
- 4. 多重检验：当检验大量分类群时，假阳性风险增加，应使用校正后的 P 值

5.6 小提示

microeco 包的使用文档参见：https://chiliubio.github.io/microeco_tutorial。

- 运行 `tidy_taxonomy()` 是差异分析前的必要步骤，否则可能导致 `cladogram` 绘图失败
- `filter_thres` 参数用于预先过滤低丰度分类群，减少多重检验负担
- 使用多种差异分析方法并取交集，可以提高结果的可靠性
- `plot_diff_abund()` 中的 `add_sig = TRUE` 可以添加显著性标记

第 6 章 网络分析

6.1 概述

微生物共现网络（Co-occurrence Network）分析通过推断微生物类群间的统计关联，揭示微生物群落内的潜在相互作用关系。网络分析将复杂的群落数据转化为直观的图结构，节点代表微生物类群，边代表类群间的关联（正相关或负相关）。

6.1.1 网络分析的生态学意义

- 相互作用推断：正相关可能暗示互利共生、交叉喂养或共同的环境偏好；负相关可能暗示竞争、拮抗或生态位分化
- 关键物种识别：发现网络中的核心节点（Hub nodes）和关键物种（Keystone species）
- 群落稳定性：通过网络拓扑特征评估群落的稳定性和鲁棒性
- 功能模块：识别功能相关的微生物模块（Modules）

6.1.2 网络构建方法

方法	原理	特点
相关性网络	Spearman/Pearson 相关	简单直观，计算快速
SparCC	稀疏相关性	校正组成性效应
SPIEC-EASI	稀疏逆协方差	考虑条件独立性，去除间接效应
WGCNA	加权相关网络	适合发现模块结构

6.1.3 网络拓扑指标

指标	含义	生态学解释
度（Degree）	节点的连接数	连接度高的可能是关键物种
介数中心性（Betweenness）	作为桥梁的频率	控制信息流动的关键节点
模块内连接度（Zi）	在模块内的连接程度	模块核心或外围
模块间连接度（Pi）	与其他模块的连接程度	网络连接器或模块特化者

6.1.4 节点分类（基于 Zi 和 Pi）

- 网络枢纽（Network hubs）： $Z_i > 2.5$, $P_i > 0.62$ ，在模块内外都高度连接
- 模块枢纽（Module hubs）： $Z_i > 2.5$, $P_i < 0.62$ ，主要连接模块内部
- 连接器（Connectors）： $Z_i < 2.5$, $P_i > 0.62$ ，连接不同模块
- 外围节点（Peripherals）： $Z_i < 2.5$, $P_i < 0.62$ ，大部分节点属于此类

6.2 流程

在 microeco 包中，网络分析通过 trans_network 类实现，主要流程如下：

1. 初始化对象：指定相关计算方法和过滤阈值
2. 构建网络：计算相关性并构建网络
3. 模块划分：使用聚类算法识别网络模块
4. 拓扑分析：计算网络属性和节点角色
5. 可视化：绘制网络图并导出到外部软件

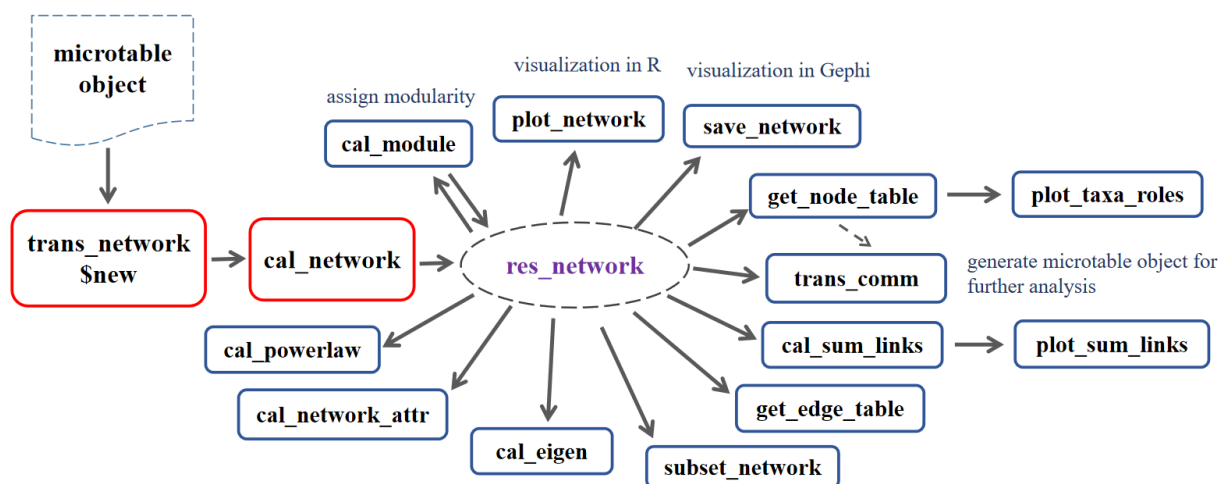


图 6.1: 使用 microeco 包进行微生物网络分析的流程

6.3 构建 microtable 对象

这里使用 microeco 软件包进行数据分析。

首先需要从数据保存的目录读取数据，并构建 microtable 对象。

```
PATH = "data/amplicon/processed"
list.files(PATH, full.names = TRUE)

#> [1] "data/amplicon/processed/asv.fasta"
#> [2] "data/amplicon/processed/asvtab.csv"
#> [3] "data/amplicon/processed/metadata.csv"
#> [4] "data/amplicon/processed/taxtab.csv"

metadata_file = file.path(PATH, "metadata.csv")
otu_table_file = file.path(PATH, "asvtab.csv")
tax_table_file = file.path(PATH, "taxtab.csv")
sequence_file = file.path(PATH, "asv.fasta")
```

6.3.1 载入包

```
library(tidyverse)
library(microeco)

theme_set(theme_bw())
```

6.3.2 OTU 表

```
otu_table = read_csv(otu_table_file)
otu_table = column_to_rownames(otu_table, var = names(otu_table)[1])
```

在 microeco 包（以及大多数 R 语言生物信息包）的规范中，microtable 对 otu_table 的格式有严格要求：行必须是 **Taxa** (ASV/OTU)，列必须是 **Samples**。

如果你的数据正好反过来了（行是样品，列是 ASV），需要在传入 otu_table 之前，使用 `t()` 函数

对其进行转置，并确保它仍然是一个 `data.frame`。

```
otu_table <- as.data.frame(t(otu_table))
```

为了让你直观理解 `microtable` 的结构逻辑，可以参考下图的对应关系：

- **OTU Table**: 行名必须对接 **Taxonomy Table** 的行名（即 ASV ID）。
- **OTU Table**: 列名必须对接 **Sample Table** 的行名（即 Sample ID）。

6.3.3 Sample 表

```
sample_table = read_csv(metadata_file)
sample_table = column_to_rownames(sample_table, var = names(sample_table)[1])
```

6.3.4 Taxonomy 表

```
taxon_table = read_csv(tax_table_file)
taxon_table = column_to_rownames(taxon_table, var = names(taxon_table)[1]) |>
  mutate(Genus = if_else(str_detect(Genus, "Burkholderia"), "Burkholderia", Genus))
```

6.3.5 构建对象

构建 `microtable` 对象。

```
mt = microtable$new(otu_table = otu_table, sample_table = sample_table, tax_table = taxon_table)
mt
```

```
#> microtable-class object:
```

```
#> sample_table have 12 rows and 8 columns
```

```
#> otu_table have 7230 rows and 12 columns
```

```
#> tax_table have 7230 rows and 7 columns
```

处理 Taxonomic 表格中的 NA 值。

- 删掉 `Phylum == NA` 的 ASV（根本没比对上的垃圾序列）。

```
mt$tax_table = subset(mt$tax_table, !is.na(Phylum))
mt$tidy_dataset()
```

- 为未知的 Genus 填充数据（自上级非 NA）。

```
# 该函数会自动查找分类表中的缺失值并填充
```

```
mt$tax_table <- tidy_taxonomy(mt$tax_table)
```

```
# 填充完成后，运行 tidy_dataset 同步一下
```

```
mt$tidy_dataset()
```

6.4 实现

6.4.1 相关性网络构建

使用 Spearman 相关性构建共现网络是最常用的方法。

```
# 初始化 trans_network 对象
```

```
t1 = trans_network$new(
  dataset = mt,
  cor_method = "spearman",
  filter_thres = 0.001,
  tax_level = "Genus"
)
```

参数说明:

- `cor_method = "spearman"`: 使用 Spearman 秩相关 (对非正态分布稳健)
- `filter_thres = 0.001`: 过滤相对丰度 $< 0.1\%$ 的分类群
- `tax_level = "Genus"`: 在属水平构建网络

6.4.2 计算网络

```
# 构建网络, 设置相关性阈值
t1$cal_network(
  COR_p_thres = 0.01,
  COR_cut = 0.6,
  COR_optimization = FALSE
)

# 查看网络基本信息
t1$res_network

#> IGRAPH 5c9e670 UNW- 49 503 --
#> + attr: name (v/c), taxa (v/c), Phylum (v/c), RelativeAbundance (v/n),
#> | label (e/c), weight (e/n)
#> + edges from 5c9e670 (vertex names):
#> [1] ASV5f2cb69b4d62--ASV8d45d97f57c1 ASV5f2cb69b4d62--ASV80a264755ead
#> [3] ASV5f2cb69b4d62--ASV3b5c3206737e ASV5f2cb69b4d62--ASVfb6534e8d2a0
#> [5] ASV5f2cb69b4d62--ASV3fb2a0fcae7d ASV5f2cb69b4d62--ASV86c713b402b1
#> [7] ASV5f2cb69b4d62--ASVa75e4fe1b306 ASV5f2cb69b4d62--ASV6e51124543c4
#> [9] ASV5f2cb69b4d62--ASV364507f2c1af ASV5f2cb69b4d62--ASV123596181742
#> [11] ASV5f2cb69b4d62--ASVb4604014a1ec ASV5f2cb69b4d62--ASV3e7d2a4331d6
#> [13] ASV5f2cb69b4d62--ASV86ce01c03c9c ASV5f2cb69b4d62--ASVcf135dfce25c
#> + ... omitted several edges
```

参数说明:

- `COR_p_thres = 0.01`: P 值阈值, 仅保留 $P < 0.01$ 的相关性
- `COR_cut = 0.6`: 相关系数阈值, 仅保留 $|r| > 0.6$ 的边
- `COR_optimization = TRUE` 可基于随机矩阵理论自动优化阈值

6.4.3 模块划分

使用社区检测算法将网络划分为功能模块。

```
# 使用 fast greedy 算法划分模块
t1$cal_module(method = "cluster_fast_greedy")

# 查看模块划分结果
table(t1$res_node_table$module)

#> < table of extent 0 >
```

6.4.4 网络可视化

6.4.4.1 在 R 中绘制网络图

```
require(ggraph)
# 使用 ggraph 方法绘制网络
t1$plot_network(
  method = "ggraph",
  node_color = "module",
```



```
node_label = "Genus",
node_size_range = c(3, 10)
)
```

```
#> Error in `alg_fun()`:
#> ! unused argument (node_size_range = c(3, 10))
```

图形元素说明：

- 节点颜色：所属模块
- 节点大小：连接度 (Degree)
- 边的颜色：正相关 (通常) vs 负相关
- 边的粗细：相关性强弱

6.4.4.2 导出到 Gephi

对于 publication-quality 的图形，建议导出到 Gephi 进行美化。

```
# 保存为 GEXF 格式 (需要 rgexf 包)
t1$save_network(filepath = "network.gexf")
```

6.4.5 网络拓扑分析

6.4.5.1 计算网络属性

```
# 计算网络拓扑属性
t1$cal_network_attr()
t1$res_network_attr

#>
#> Vertex          49.0000000
#> Edge            503.0000000
#> Average_degree   20.5306122
#> Average_path_length 1.0757741
#> Network_diameter 3.0000000
#> Clustering_coefficient 0.9454859
#> Density          0.4277211
#> Heterogeneity    0.5377284
#> Centralization   0.2181122
#> Modularity       0.1891820
```

关键指标解读：

- 平均度 (Average degree)：每个节点的平均连接数
- 聚类系数 (Clustering coefficient)：网络的聚集程度
- 平均路径长度：网络中任意两节点间的平均最短距离
- 模块化指数 (Modularity)：模块划分的质量 (>0.4 表示显著模块化)

6.4.5.2 节点角色分析

```
# 获取节点属性表
t1$get_node_table(node_roles = TRUE)
head(t1$res_node_table)

#>
#> name degree betweenness centrality
#> ASV5f2cb69b4d62 ASV5f2cb69b4d62 29 0
#> ASV8d45d97f57c1 ASV8d45d97f57c1 30 0
#> ASV80a264755ead ASV80a264755ead 29 0
#> ASV8959eb50740b ASV8959eb50740b 3 0
#> ASVb03150ca3ed0 ASVb03150ca3ed0 10 1
```

```

#> ASVa4d6e43a78d0 ASVa4d6e43a78d0      3      0
#>      closeness centrality eigenvector centrality Abundance module
#> ASV5f2cb69b4d62      0.02864384      0.95417502 0.2344011      M1
#> ASV8d45d97f57c1      0.03063871      0.98641443 0.2151080      M1
#> ASV80a264755ead      0.03040144      0.89085676 0.1862587      M1
#> ASV8959eb50740b      0.01626549      0.02469697 0.1294615      M1
#> ASVb03150ca3ed0      0.11147773      0.00000000 0.1597533      M2
#> ASVa4d6e43a78d0      0.01606384      0.02509474 0.1186430      M1
#>      z p      taxa_roles      Kingdom      Phylum
#> ASV5f2cb69b4d62 0.3514032 0 Peripheral nodes k__Bacteria p__Gemmatimonadota
#> ASV8d45d97f57c1 0.4871726 0 Peripheral nodes k__Bacteria p__Gemmatimonadota
#> ASV80a264755ead 0.3514032 0 Peripheral nodes k__Bacteria p__Gemmatimonadota
#> ASV8959eb50740b -3.1786013 0 Peripheral nodes k__Bacteria p__Pseudomonadota
#> ASVb03150ca3ed0 0.5838742 0 Peripheral nodes k__Bacteria p__Chloroflexota
#> ASVa4d6e43a78d0 -3.1786013 0 Peripheral nodes k__Bacteria p__Pseudomonadota
#>      Class      Order
#> ASV5f2cb69b4d62 c__Gemmatimonadia o__Gemmatimonadales
#> ASV8d45d97f57c1 c__Gemmatimonadia o__Gemmatimonadales
#> ASV80a264755ead c__Gemmatimonadia o__Gemmatimonadales
#> ASV8959eb50740b c__Gammaproteobacteria o__Lysobacterales
#> ASVb03150ca3ed0 c__Ktedonobacteria o__Ktedonobacterales
#> ASVa4d6e43a78d0 c__Gammaproteobacteria o__Lysobacterales
#>      Family      Genus      Species
#> ASV5f2cb69b4d62 f__Gemmatimonadaceae g__ s__
#> ASV8d45d97f57c1 f__Gemmatimonadaceae g__ s__
#> ASV80a264755ead f__Gemmatimonadaceae g__ s__
#> ASV8959eb50740b f__Rhodanobacteraceae g__Rhodanobacter s__spathiphylli
#> ASVb03150ca3ed0 f__JG30-KF-AS9 g__ s__
#> ASVa4d6e43a78d0 f__Rhodanobacteraceae g__Rhodanobacter s__

```

6.4.5.3 绘制节点角色图

```

# 绘制节点角色分类图
t1$plot_taxa_roles(use_type = 1)

```

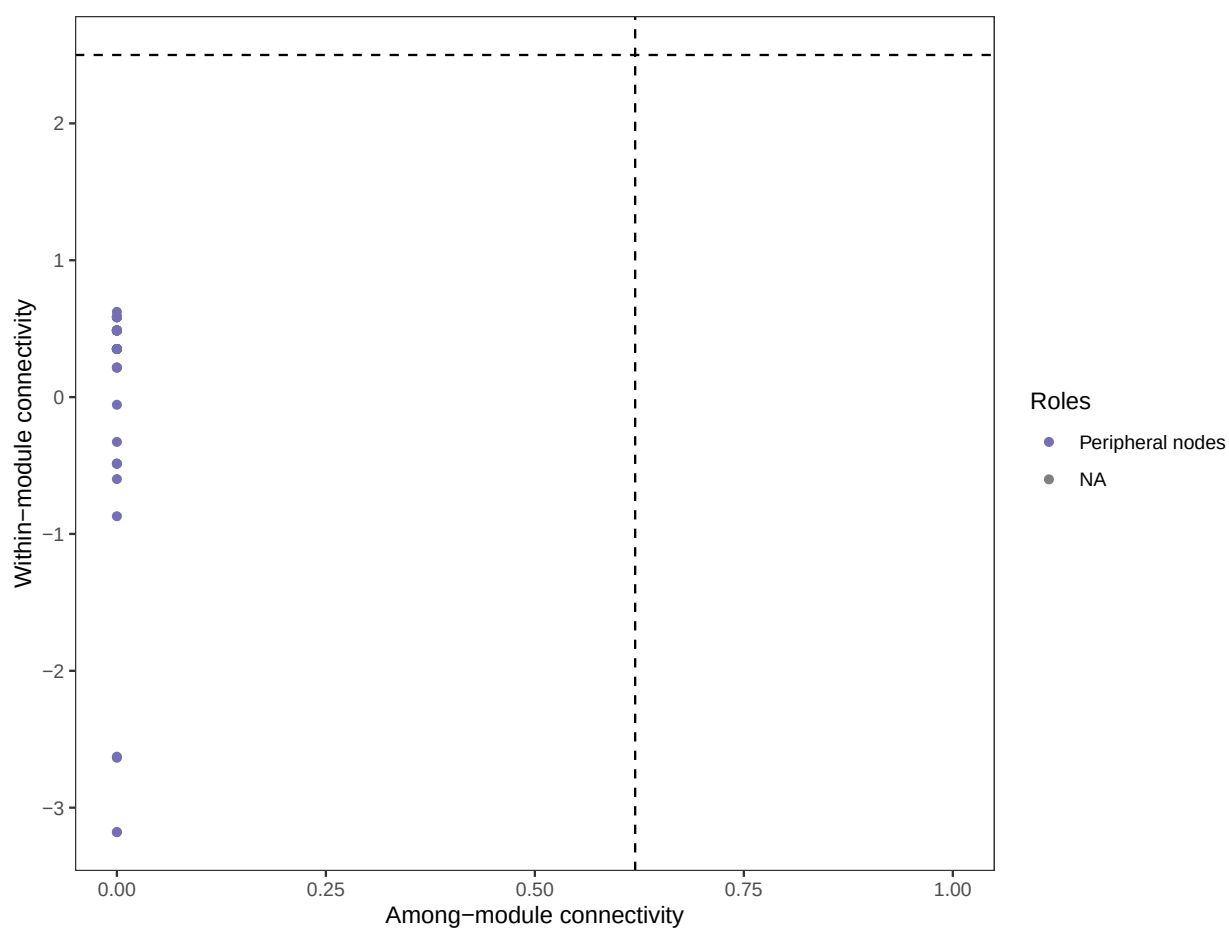


图 6.2: 节点分类图 (基于模块内和模块间连通性)

图形解读: - X 轴 (P_i): 模块间连通性, 值越大表示与其他模块连接越多 - Y 轴 (Z_i): 模块内连通性, 值越大表示在模块内部越核心 - 虚线将节点分为四类: 网络枢纽、模块枢纽、连接器和外围节点

6.4.5.4 按分类学信息绘制

```
t1$plot_taxa_roles(use_type = 2)
```

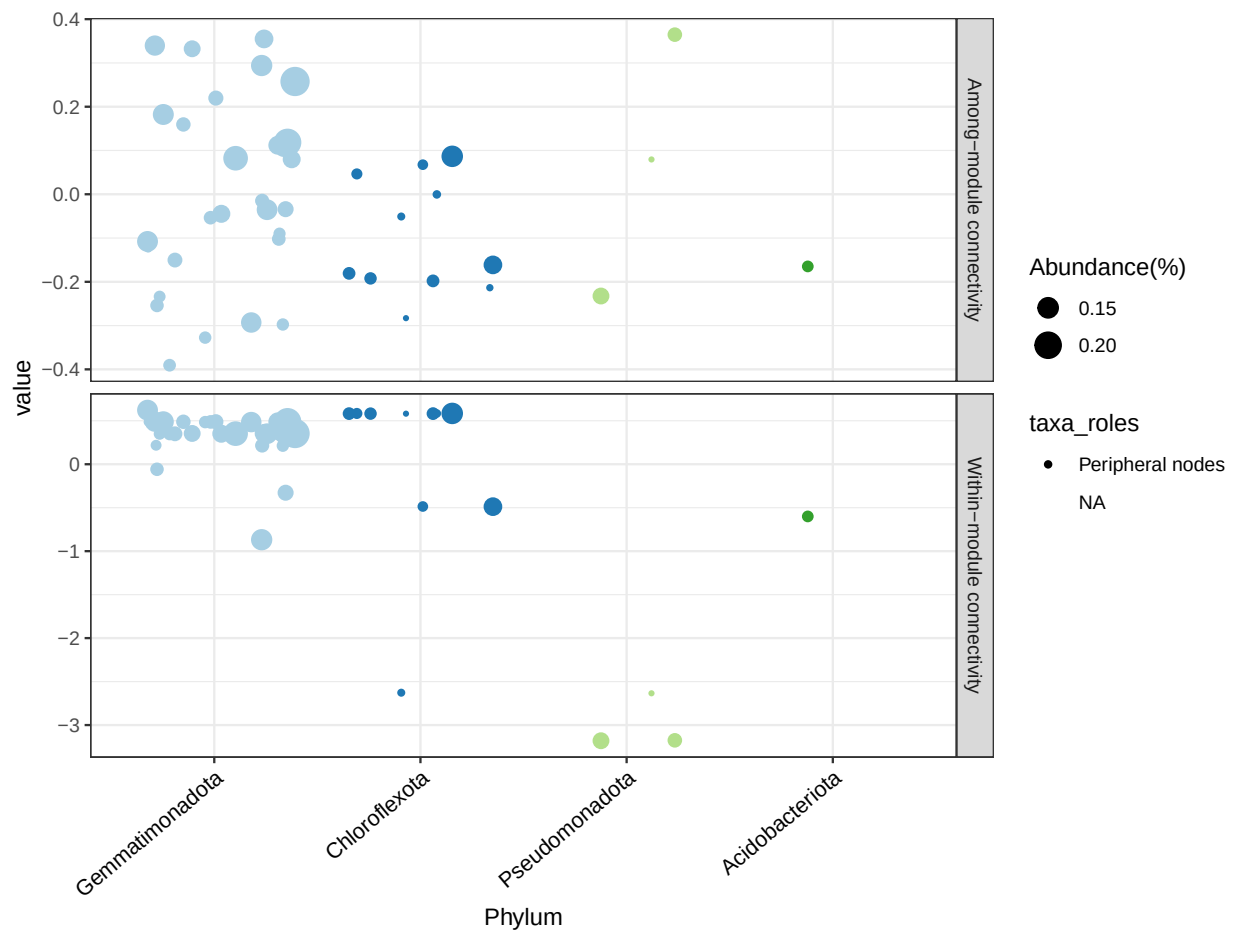


图 6.3: 节点分类图（按门水平着色）

6.4.6 边和邻接矩阵

```
# 获取边的属性
t1$get_edge_table()
head(t1$res_edge_table)

#>           node1           node2 label  weight
#> 1 ASV5f2cb69b4d62 ASV8d45d97f57c1    + 0.9560647
#> 2 ASV5f2cb69b4d62 ASV80a264755ead    + 0.8266212
#> 3 ASV5f2cb69b4d62 ASV3b5c3206737e    + 0.7915392
#> 4 ASV5f2cb69b4d62 ASVfb6534e8d2a0    + 0.9228070
#> 5 ASV5f2cb69b4d62 ASV3fb2a0fcae7d    + 0.9017544
#> 6 ASV5f2cb69b4d62 ASV86c713b402b1    + 0.9017544

# 获取邻接矩阵
t1$get_adjacency_matrix()
```

6.4.7 子网络提取

可以提取特定模块或节点的子网络进行进一步分析。

```
# 提取模块 M1 的子网络
sub_network <- t1$subset_network(
  node = t1$res_node_table %>%
    base::subset(module == "M1") %>%
    rownames,
```

```

    rm_single = TRUE
  )

# 查看子网络
sub_network

#> IGRAPH 94147b1 UNW- 34 449 --
#> + attr: name (v/c), taxa (v/c), Phylum (v/c), RelativeAbundance (v/n),
#> | module (v/c), label (e/c), weight (e/n)
#> + edges from 94147b1 (vertex names):
#> [1] ASV5f2cb69b4d62--ASV8d45d97f57c1 ASV5f2cb69b4d62--ASV80a264755ead
#> [3] ASV5f2cb69b4d62--ASV3b5c3206737e ASV5f2cb69b4d62--ASVfb6534e8d2a0
#> [5] ASV5f2cb69b4d62--ASV3fb2a0fcae7d ASV5f2cb69b4d62--ASV86c713b402b1
#> [7] ASV5f2cb69b4d62--ASVa75e4fe1b306 ASV5f2cb69b4d62--ASV6e51124543c4
#> [9] ASV5f2cb69b4d62--ASV364507f2c1af ASV5f2cb69b4d62--ASV123596181742
#> [11] ASV5f2cb69b4d62--ASVb4604014a1ec ASV5f2cb69b4d62--ASV3e7d2a4331d6
#> [13] ASV5f2cb69b4d62--ASV86ce01c03c9c ASV5f2cb69b4d62--ASVcf135dfce25c
#> + ... omitted several edges

```

6.4.8 分类群间连接分析

计算不同分类群之间的连接数量。

```

# 计算门水平间的连接数
t1$cal_sum_links(taxa_level = "Phylum")

# 绘制弦图 (需要 chorddiag 包)
t1$plot_sum_links(
  plot_pos = TRUE,
  plot_num = 10,
  color_values = RColorBrewer::brewer.pal(10, "Paired")
)

```

6.4.9 其他网络构建方法

6.4.9.1 Bray-Curtis 网络

```

t1 <- trans_network$new(
  dataset = mt,
  cor_method = "bray",
  filter_thres = 0.001
)
t1$cal_network(network_method = "COR")
t1$cal_module()
t1$plot_network(node_color = "Phylum", node_label = "Genus")

```

6.4.9.2 WGCNA 方法

```

library(WGCNA)

t1 <- trans_network$new(
  dataset = mt,
  cor_method = "spearman",
  use_WGCNA_pearson_spearman = TRUE,
  filter_thres = 0.001
)

```

```
)  
t1$cal_network()  
t1$cal_module()  
t1$plot_network()
```

6.5 结果

6.5.1 网络整体特征

基于网络拓扑属性的分析结果：

网络规模：- 节点数：网络中包含的分类群数量 - 边数：显著相关的类群对数 - 连接密度：边数与理论最大边数的比值

网络结构：- 模块化程度：高模块化指数 (>0.4) 表明网络具有显著的模块结构，暗示存在功能分化的微生物群组 - 小世界特性：高聚类系数和短平均路径长度表明网络具有小世界特性，信息传播效率高 - 度分布：符合幂律分布（无标度网络）表明存在少数高度连接的枢纽节点

6.5.2 关键节点识别

基于节点角色分析的结果：

模块枢纽 (**Module hubs**):

- 在各自模块内部高度连接
- 可能是模块内的核心功能执行者
- 对模块的稳定性具有重要影响

连接器 (**Connectors**):

- 连接不同模块的桥梁节点
- 促进模块间的物质和信息交换
- 对维持网络整体连通性至关重要

网络枢纽 (**Network hubs**) (如果存在):

- 在模块内外都高度连接
- 可能是群落中的关键物种 (**Keystone species**)
- 其丧失可能对群落功能产生重大影响

6.5.3 模块的功能推断

模块的生态学意义：

1. 功能特化：

- 同一模块内的分类群可能具有相似的功能特征或生态位偏好
- 不同模块可能执行不同的生态系统功能

2. 环境响应：

- 比较不同处理组的网络模块组成，可以识别对处理敏感的微生物群组
- 模块丰度或结构的变化可能反映功能群落的重组

3. 稳定性分析：

- 模块内部的高连接度可能增强群落的稳定性
- 模块间的连接可能影响群落的恢复力和适应性

6.5.4 网络与处理效应的关系

可能的发现模式：

1. 网络复杂度的变化：

- 若某处理组的网络边数减少，可能表明处理简化了群落结构
- 若连接度增加，可能表明处理促进了微生物间的相互作用

2. 模块重组：

- 处理可能导致分类群在网络中重新组织，形成新的模块结构

- 某些模块可能在特定处理中消失或新出现
3. 关键节点的变化:
 - 不同处理组可能具有不同的网络枢纽
 - 关键物种的转换可能反映群落功能状态的变化

6.5.5 与其他分析的关联

与差异分析的联系:

- 在网络中处于核心位置（高连接度）的分类群，如果在差异分析中也显著富集，则更可能是关键功能类群
- 连接器节点即使丰度不高，也可能通过影响模块间交流而发挥重要作用

与 Beta 多样性的联系:

- 网络拓扑的差异可以解释 Beta 多样性观察到的模式
- 组间网络结构的差异可能比单纯的组成差异更能反映功能变化

6.5.6 研究局限性与注意事项

1. 相关性不等于因果:
 - 网络中的边仅表示统计关联，不代表直接的生物学相互作用
 - 相关可能是由共同的环境偏好或间接效应引起的
2. 组成性效应:
 - 基于相对丰度的相关性分析存在组成性效应
 - 建议使用 SparCC 或 SPIEC-EASI 等方法进行验证
3. 阈值依赖性:
 - 网络结构对相关性和 P 值阈值敏感
 - 建议使用不同阈值进行敏感性分析
4. 时间动态:
 - 横截面数据构建的网络无法反映相互作用的动态变化
 - 时间序列分析可以提供更丰富的信息

6.6 小提示

microeco 包的使用文档参见: https://chiliubio.github.io/microeco_tutorial。

- 网络构建前建议先过滤低丰度分类群 (filter_thres)，以减少噪声和计算量
- COR_optimization = TRUE 可基于随机矩阵理论自动寻找最佳相关性阈值
- 使用 Gephi 软件可以获得更美观的网络可视化效果
- 节点角色分类基于 Guimerà & Amaral (2005) 的方法，广泛应用于生态网络研究

第 7 章 总结

本讲系统介绍了使用 R 语言进行微生物组数据分析的完整流程，从原始测序数据预处理到复杂的生态网络分析。通过学习本讲内容，您应该已经掌握了以下核心技能：

7.1 核心内容回顾

7.1.1 测序数据预处理 (DADA2)

- 学习了 DADA2 分析流程：质量控制、错误率学习、去噪、序列拼接、去除嵌合体、分类注释
- 理解了 ASV (Amplicon Sequence Variants) 相比传统 OTU 聚类的优势
- 掌握了使用 SILVA 数据库进行物种注释的方法

关键点：

- 测序质量控制是确保数据可靠性的第一步
- 错误率学习是 DADA2 实现高精度去噪的核心
- ASV 提供了单核苷酸分辨率的物种区分能力

7.1.2 构建 microtable 对象

- 学会了将各类数据 (OTU/ASV 表、样本表、分类表) 整合为 microtable 对象
- 掌握了数据清洗和格式标准化的方法
- 理解了 microeco 包的数据结构和工作原理

关键点：

- microtable 是 microeco 包分析的基础数据结构
- 数据格式的标准化 (行 = 特征, 列 = 样本) 至关重要
- tidy_taxonomy() 和 tidy_dataset() 是数据预处理的关键步骤

7.1.3 群落结构分析

- 使用 trans_abund 类进行物种组成的可视化
- 掌握了柱状图、热图、箱线图等多种可视化方法
- 学会了在不同分类等级 (门、纲、目、科、属) 分析群落组成

关键点：- 多分类等级的分析有助于从不同分辨率理解群落 - 可视化是发现科学问题的起点 - 相对丰度数据需要注意组成性效应

7.1.4 Alpha 多样性分析

- 理解了 Alpha 多样性的生态学意义和常用指标
- 学会了数据抽平 (Rarefaction) 的方法
- 掌握了组间差异检验和可视化的方法

关键点：

- Observed/Chao1 反映物种丰富度, Shannon/Simpson 反映综合多样性
- 抽平是消除测序深度差异影响的标准方法
- 统计方法的选择应根据数据分布特征决定

7.1.5 Beta 多样性分析

- 理解了 Beta 多样性的概念和常用距离指标
- 掌握了 PCoA 和 NMDS 排序分析方法
- 学会了 PERMANOVA 等统计检验方法

关键点：

- Bray-Curtis 是生态学中最常用的距离指标
- 排序分析可以直观展示样本间的相似性关系

- PERMANOVA 是检验组间差异显著性的标准方法

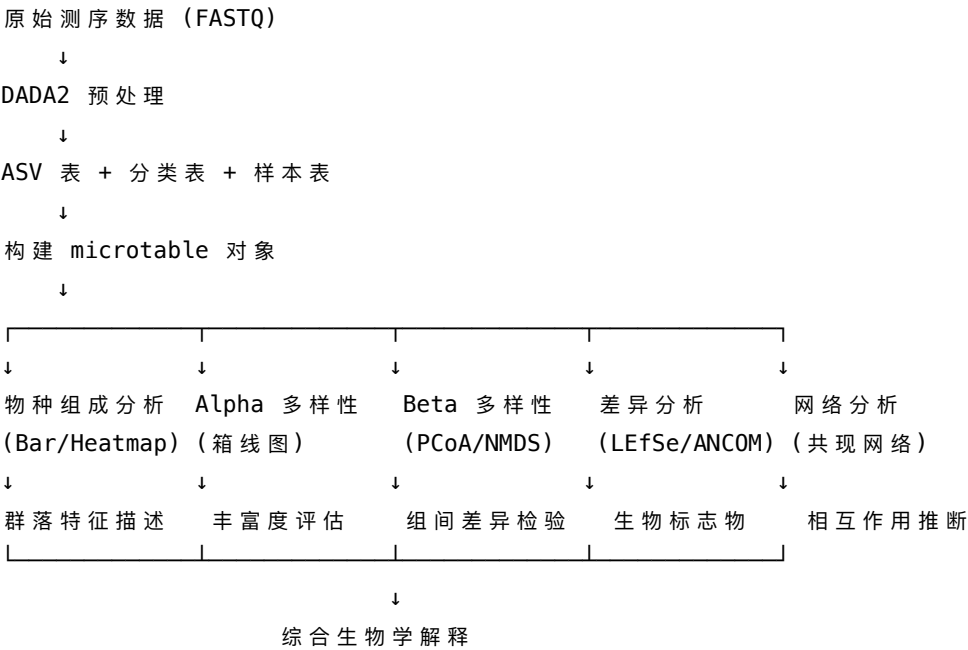
7.1.6 差异分析

- 学习了多种差异分析方法（LEfSe、ANCOM-BC2、随机森林等）
 - 理解了不同方法的特点和适用场景
 - 掌握了生物标志物的识别和可视化
- 关键点：
- 不同方法可能产生不同结果，建议多种方法互相验证
 - ANCOM-BC2 是当前推荐的校正组成性效应的方法
 - 统计显著性不等于生物学重要性，需要结合功能知识解释

7.1.7 网络分析

- 理解了微生物共现网络的生态学意义
 - 学会了网络构建、模块划分和拓扑分析
 - 掌握了节点角色分类和关键物种识别
- 关键点：
- 网络分析可以揭示物种间的潜在相互作用
 - 模块结构反映了功能相关的微生物群组
 - 网络枢纽可能是群落中的关键物种

7.2 分析流程总结图



7.3 关键注意事项

7.3.1 数据质量

1. 质量控制：始终从测序质量评估开始，确保数据可靠性
2. 数据过滤：适当过滤低丰度特征，减少噪声和多重检验负担
3. 数据标准化：根据分析类型选择适当的标准化方法（抽平、CLR 等）

7.3.2 统计分析

1. 方法选择：根据数据分布和研究问题选择合适的统计方法
2. 多重检验：进行多重比较时注意校正 P 值，控制假阳性率
3. 效应量：关注效应量大小，不仅依赖 P 值判断生物学意义

7.3.3 结果解释

1. 组成性效应：相对丰度数据存在组成性效应，解释差异时需谨慎
2. 相关性 $\neq$ 因果性：网络分析中的相关关系不证明直接相互作用
3. 功能推断：基于分类学的功能推断是间接的，建议通过功能预测或实验验证

7.4 拓展学习方向

1. 功能预测分析：
 - 使用 PICRUST2 预测微生物群落的功能潜能
 - 结合代谢通路分析深入理解群落功能
2. 时间序列分析：
 - 如果拥有时间序列数据，可分析群落的动态变化
 - 使用 `mecoturn` 包进行群落转变分析
3. 多组学整合：
 - 结合宏基因组、代谢组等数据进行全面分析
 - 使用 `trans_env` 类分析微生物-环境因子关系
4. 高级统计方法：
 - 学习混合效应模型处理复杂实验设计
 - 探索贝叶斯方法在微生物组分析中的应用
5. 可重复性研究：
 - 使用 `renv` 包管理分析环境
 - 编写可重复的 R Markdown/Quarto 文档
 - 将分析流程封装为可重用的脚本或包

7.5 推荐资源

7.5.1 文档与教程

- [microeco 官方教程](#)
- [DADA2 官方文档](#)
- [QIIME 2 文档](#)

7.5.2 重要文献

1. Callahan et al. (2016). DADA2: High-resolution sample inference from Illumina amplicon data. *Nature Methods*
2. McMurdie & Holmes (2013). phyloseq: An R package for reproducible interactive analysis and graphics of microbiome census data. *PLoS ONE*
3. Mallick et al. (2021). Experimental design and quantitative analysis of microbial community multiomics. *Genome Biology*

7.5.3 R 包资源

- **microeco**: 微生物组全面分析
- **phyloseq**: 微生物组数据结构和管理
- **vegan**: 生态学统计分析
- **ggplot2**: 数据可视化

7.6 结语

微生物组数据分析是一个快速发展的领域，新的方法和工具不断涌现。掌握本讲介绍的基础分析流程后，建议保持对最新文献和方法的关注，根据研究问题的需要不断学习新的技能。

记住，数据分析的目的是回答科学问题，而非仅仅运行代码。在每一步分析中，都应该思考：

- 这个分析要解决什么生物学问题？
- 结果是否符合生态学原理？

- 如何验证和深化初步发现?

希望本讲能够为您的微生物组研究打下坚实基础，祝您科研顺利！

References

Knuth (1984)

Knuth, Donald E. 1984. “Literate Programming.” *Comput. J.* 27 (2): 97–111. <https://doi.org/10.1093/comjnl/27.2.97>.